WELCOME TO VUEDU

Our website offers multiple features that help students make their studies easier. We provide free all-subject handouts, MCQs files, and live quiz tests for all subjects.



Join Our WhatsApp Group:

AI-powered VU group that provides all-subject handouts, MCQs files, and free help in just 4 seconds

FREE ALL SUBJECTS HANDOUTS, MCQS FILES, AND QUIZ TEST POWERED BY VUEDU

VUEDU QUIZ TEST

Cs304 Handouts Finals Mcqs

Q1. How does the compiler distinguish between pre-increment and post-increment operator overloading?

A. By return type

B. By dummy int argument

C. By function name

D. By class access

Difficulty: Medium | Importance: 5/5

Explanation:
A dummy int argument is used in the function signature to indicate post-increment overloading.

Q2. In C++, which keyword is used to restrict automatic implicit type conversions via constructors?

A. const

B. static

C. explicit

D. virtual

Difficulty: Medium | Importance: 5/5

Explanation:
The explicit keyword prevents the compiler from performing implicit conversions using single-argument constructors.

Q3. When converting a user-defined class to a basic type (e.g., int), which syntax is correct?

A. int operator()

B. operator int()

C. int operator int()

D. void operator int()

Difficulty: Hard | Importance: 4/5

Explanation:
Conversion operators do not specify a return type in their declaration; it is implicitly taken as the type name.

Q4. In inheritance, the relationship between a derived class and a base class is best described as:

A. HAS-A

B. IS-A

C. USES-A

D. PART-OF

Difficulty: Easy | Importance: 5/5

Explanation:
Inheritance represents an 'IS-A' relationship (e.g., a Student IS A Person).

Q5. What is the order of constructor execution when an object of a derived class is created?

A. Derived first, then Base

B. Base first, then Derived

C. Random order

D. Simultaneous execution

Difficulty: Medium | Importance: 5/5

Explanation:
The base class constructor is executed first to initialize the anonymous base object before the derived class constructor.

Q6. Which access specifier ensures that members are accessible in the derived class but not from outside?

A. private

B. public

C. protected

D. friend

Difficulty: Medium | Importance: 4/5

Explanation:
Protected members are accessible in derived classes but remain private to the outside world.

Q7. When overloading the pre-increment operator (++h), what should generally be returned?

A. A copy of the object

B. void

C. A reference to the object

D. An integer code

Difficulty: Medium | Importance: 4/5

Explanation:
Returning a reference allows the object to be used as an lvalue in expressions.

Q8. In UML notation, how is inheritance represented?

A. Dotted line to derived

B. Arrow from derived to base

C. Arrow from base to derived

D. Solid line with diamond

Difficulty: Easy | Importance: 3/5

Explanation:
An arrow pointing from the derived class to the parent (base) class represents inheritance in UML.

Q9. To convert a basic type (like int) to a class type, what is required?

A. Operator overloading

B. A constructor with one parameter

C. A destructor

D. A friend function

Difficulty: Medium | Importance: 4/5

Explanation:
A constructor taking a single argument of the basic type is used to convert that type into a class object.

Q10. Are private members of a base class accessible directly in the derived class?

A. Yes, always

B. No, never

C. Only if static

D. Only if virtual

Difficulty: Easy | Importance: 5/5

Explanation:
Private members of the base class are hidden from the derived class to maintain information hiding.

Q11. In post-increment operator overloading, what value is returned?

A. The incremented value

B. The old value

C. Null

D. Reference to current object

Difficulty: Medium | Importance: 4/5

Explanation:
In post-increment, the current (old) value is saved in a temporary variable and returned after incrementing the object.

Q12. What mechanism does the compiler use to automatically convert compatible types (e.g., float to int)?

A. Type coercion

B. Dynamic casting

C. Polymorphism

D. Encapsulation

Difficulty: Medium | Importance: 3/5

Explanation:
The compiler automatically performs type coercion for compatible basic types, sometimes issuing a warning.

Q13. Which function is used in the text to convert an integer to a string inside a constructor?

A. atoi

B. itoa

C. strcat

D. strcpy

Difficulty: Hard | Importance: 3/5

Explanation:
The itoa function is used to convert an integer to a character array (string).

Q14. Every object of a derived class contains an anonymous object of which class?

A. Global class

B. Friend class

C. Base class

D. Virtual class

Difficulty: Easy | Importance: 3/5

Explanation:
Memory allocation for a derived class object includes an anonymous instance of the base class.

Q15. How many arguments does a non-member function require to overload the post-increment operator?

A. Zero

B. One

C. Two

D. Three

Difficulty: Hard | Importance: 3/5

Explanation:
A non-member function for post-increment requires two arguments: one for the object and one dummy int.

Q16. Which of the following creates a dynamic array of characters?

A. char name[30];

B. char * name = new char[30];

C. char name = new char;

D. string name;

Difficulty: Medium | Importance: 3/5

Explanation:
The new operator is used to allocate memory dynamically for a character array.

Q17. Why is explicit keyword NOT used with type conversion operators (like operator int())?

A. It is strictly for constructors

B. It makes them static

C. It causes runtime errors

D. It is valid but useless

Difficulty: Medium | Importance: 4/5

Explanation:
The text specifies that the explicit keyword only works with constructors to prevent implicit conversion.

Q18. In the context of inheritance, the parent class is also known as:

A. Sub class

B. Derived class

C. Base class

D. Child class

Difficulty: Easy | Importance: 5/5

Explanation:
The class whose behavior is inherited is called the base class or parent class.

Q19. What is a major drawback of automatic implicit type conversion via constructors?

A. Slower performance

B. Memory leaks

C. Unintended conversions

D. Syntax errors

Difficulty: Medium | Importance: 4/5

Explanation:
Automatic conversion can happen transparently even when not intended, potentially leading to logical errors (e.g., int treated as ASCII).

Q20. In which order are destructors called when an object of a derived class is destroyed?

A. Base then Derived

B. Derived then Base

C. Random order

D. Simultaneously

Difficulty: Medium | Importance: 5/5

Explanation:
Destructors are called in the reverse order of constructors: the derived class destructor executes first, followed by the base class destructor.

Q21. If a class has a user-defined constructor with parameters, what happens to the implicit default constructor?

A. It remains available

B. Compiler removes it

C. It becomes virtual

D. It becomes private

Difficulty: Medium | Importance: 5/5

Explanation:
If a user defines any constructor, the compiler stops generating the implicit default constructor.

Q22. Which access specifier allows members to be accessed in derived classes but not by the outside world?

A. private

B. public

C. protected

D. friend

Difficulty: Easy | Importance: 5/5

Explanation:
Protected members act like private members to the outside world but are accessible within derived classes.

Q23. Which syntax is used to call a base class constructor explicitly from a derived class constructor?

A. Inside constructor body

B. Member initialization list

C. Using scope resolution

D. Using super keyword

Difficulty: Medium | Importance: 4/5

Explanation:
The base class constructor is called in the member initialization list using the syntax Derived(): Base().

Q24. What is the relationship modeled by public inheritance in C++?

A. HAS-A

B. IS-A

C. USES-A

D. PART-OF

Difficulty: Easy | Importance: 5/5

Explanation:
Public inheritance models an 'IS-A' relationship, meaning the derived object is a specific kind of the base object.

Q25. Can a base class pointer hold the address of a derived class object?

A. Yes, always

B. No, never

C. Only if static

D. Only if const

Difficulty: Medium | Importance: 5/5

Explanation:
Since a derived class IS-A base class, a base class pointer can point to a derived class object.

Q26. What happens if you try to access a derived-class-specific method using a base class pointer?

A. It works fine

B. Compiler generates error

C. Runtime crash

D. Calls base method

Difficulty: Hard | Importance: 4/5

Explanation:
A base class pointer can only access members defined in the base class interface; accessing derived-specific members requires casting.

Q27. What is a primary disadvantage of using protected members in C++?

A. Slower execution

B. Breaks encapsulation

C. Increases memory usage

D. Compiler errors

Difficulty: Medium | Importance: 4/5

Explanation:
Protected members break encapsulation because they expose implementation details to derived classes.

Q28. When is the base class constructor executed relative to the derived class constructor?

A. After derived constructor

B. Before derived constructor

C. Simultaneously

D. Depending on syntax

Difficulty: Easy | Importance: 5/5

Explanation:
The base class constructor always executes before the derived class constructor to initialize the base part of the object.

Q29. How can a derived class initialize a private data member of its base class?

A. Direct assignment

B. Member initialization list

C. Calling base constructor

D. Friend function

Difficulty: Medium | Importance: 4/5

Explanation:
Derived classes cannot directly access private base members; they must pass values to the base class constructor.

Q30. What is an Explicit Default Constructor?

A. Compiler generated

B. User defined no-args

C. Virtual constructor

D. Copy constructor

Difficulty: Medium | Importance: 3/5

Explanation:
An explicit default constructor is a user-defined constructor that takes no arguments or has all default arguments.

Q31. If a base class lacks a default constructor and has only parameterized ones, what must the derived class do?

A. Do nothing

B. Define default constructor

C. Call base constructor explicitly

D. Use friend class

Difficulty: Hard | Importance: 5/5

Explanation:
The derived class must explicitly call the base class's parameterized constructor in its initialization list to avoid errors.

Q32. Helper functions that should not be exposed to the outside world are typically declared as:

A. public

B. private

C. static

D. virtual

Difficulty: Easy | Importance: 3/5

Explanation:
Helper functions used only internally by the class should be private to enforce data hiding.

Q33. Which concept determines access to member functions when using a pointer?

A. Dynamic binding

B. Static typing

C. Operator overloading

D. Template matching

Difficulty: Hard | Importance: 3/5

Explanation:
The compiler uses the static type of the pointer (e.g., Base*) to determine which members are accessible.

Q34. In the context of inheritance, can a derived class initialize public data members of the base class directly in its list?

A. Yes, always

B. No, never

C. Only if static

D. Only if const

Difficulty: Hard | Importance: 4/5

Explanation:
The text states derived classes cannot initialize base class members directly in the initialization list; they must use the base constructor.

Q35. What output is generated first when a Child object is created?

A. Child Constructor

B. Parent Constructor

C. Child Destructor

D. Parent Destructor

Difficulty: Easy | Importance: 5/5

Explanation:
The base (Parent) constructor executes first, so its output appears before the Child constructor's output.

Q36. What is the specific utility of the protected access specifier regarding helper functions?

A. Make them public

B. Hide from derived

C. Share with derived

D. Make them static

Difficulty: Medium | Importance: 4/5

Explanation:
It allows helper functions to be used by derived classes while keeping them hidden from the rest of the program.

Q37. If Child inherits from Parent, which statement is true regarding object substitution?

A. Parent is Child

B. Child is Parent

C. Child uses Parent

D. Parent has Child

Difficulty: Easy | Importance: 4/5

Explanation:
Inheritance implies that the Child object is a special kind of Parent object (IS-A relationship).

Q38. When using an initializer list, when is the base class constructor executed relative to the derived class's member initialization?

A. Before member init

B. After member init

C. Simultaneously

D. Randomly

Difficulty: Medium | Importance: 3/5

Explanation:
The base class constructor is executed before the initialization of data members of the derived class.

Q39. What does the compiler generate if no constructor is explicitly defined in a class?

A. Destructor only

B. Implicit default constructor

C. Copy constructor only

D. Nothing

Difficulty: Easy | Importance: 4/5

Explanation:
If the user provides no constructor, the compiler automatically generates an implicit default constructor.

Q40. What is the 'static type' of a pointer or reference in C++?

A. The type of the object it points to

B. The type used to declare it

C. The type determined at runtime

D. The type of the derived class

Difficulty: Medium | Importance: 5/5

Explanation:
The static type is the type used in the declaration of the identifier (e.g., in 'Person * p;', static type is Person*).

Q41. If you have Person * p = new Student();, which member functions can p access?

A. Only Student members

B. Only Person members

C. Both Person and Student members

D. None

Difficulty: Medium | Importance: 5/5

Explanation:
The compiler uses the static type (Person*) to determine member access, so only members defined in the Person class are accessible.

Q42. Which access specifier allows members to be accessed by derived classes but not by the outside world?

A. private

B. public

C. protected

D. friend

Difficulty: Easy | Importance: 5/5

Explanation:
Protected members are accessible within the class and its derived classes, but behave like private members to the outside world.

Q43. Public inheritance in C++ models which type of relationship?

A. HAS-A

B. IS-A

C. USES-A

D. PART-OF

Difficulty: Easy | Importance: 5/5

Explanation:
Public inheritance represents an IS-A relationship, meaning the derived object is a specific kind of the base object.

Q44. In the context of inheritance, what happens if a derived class copy constructor does not explicitly call the base class copy constructor?

A. Base copy constructor is called automatically

B. Base default constructor is called

C. Compiler generates an error

D. Runtime exception occurs

Difficulty: Hard | Importance: 4/5

Explanation:
If the base copy constructor is not called explicitly in the initializer list, the compiler calls the base class's default constructor.

Q45. What is the primary problem with the default compiler-generated copy constructor when a class contains pointers?

A. Syntax error

B. Shallow copy

C. Deep copy

D. Memory leak only

Difficulty: Medium | Importance: 5/5

Explanation:
The default copy constructor performs a shallow copy, meaning pointers will point to the same memory location, leading to potential double deletion errors.

Q46. To perform a 'Deep Copy' in a derived class, what must be done regarding the base class?

A. Call base default constructor

B. Call base copy constructor explicitly

C. Nothing is needed

D. Overload assignment operator

Difficulty: Hard | Importance: 4/5

Explanation:
The derived class copy constructor must explicitly invoke the base class copy constructor to ensure the base part is also deep copied.

Q47. Can a derived class object be passed to a function that expects a base class object by reference?

A. Yes, always

B. No, never

C. Only if casted

D. Only if private inheritance

Difficulty: Easy | Importance: 4/5

Explanation:
Because of the IS-A relationship, a derived class object can be treated as a base class object (upcasting).

Q48. What is the syntax to call the base class copy constructor from the derived class copy constructor?

A. Derived(const Derived &d) : Base(d)

B. Derived(const Derived &d) { Base(d); }

C. Derived(const Derived &d) : Base()

D. Derived(const Derived &d) : copy(Base)

Difficulty: Hard | Importance: 4/5

Explanation:
The base copy constructor is called in the member initialization list, passing the derived object which is compatible with the base reference.

Q49. In the provided text, why did the output show 'Name: NULL' when the derived copy constructor failed to call the base copy constructor?

A. Memory leak

B. Base default constructor set name to NULL

C. Destructor was called early

D. Shallow copy failed

Difficulty: Hard | Importance: 3/5

Explanation:
Since the base copy constructor wasn't called, the base default constructor executed, which initialized the 'name' attribute to NULL.

Q50. A copy constructor is a member function used to create an object by:

A. Initializing with zero

B. Copying from an existing object

C. Reading from file

D. Allocating random memory

Difficulty: Easy | Importance: 3/5

Explanation:
A copy constructor initializes a new object as a copy of an existing object of the same class.

Q51. When Student sobj2 = sobj1; is executed, which constructor is called?

A. Default constructor

B. Copy constructor

C. Parameterized constructor

D. Destructor

Difficulty: Medium | Importance: 4/5

Explanation:
This syntax initializes a new object sobj2 using an existing object sobj1, invoking the copy constructor.

Q52. Protected members of a base class become what type of members in a publicly derived class?

A. Private

B. Public

C. Protected

D. Static

Difficulty: Medium | Importance: 4/5

Explanation:
In public inheritance, protected members of the base class remain protected in the derived class.

Q53. Which type of copy does the compiler perform by default?

A. Deep Copy

B. Shallow Copy

C. Virtual Copy

D. Static Copy

Difficulty: Easy | Importance: 5/5

Explanation:
The compiler generates a copy constructor that performs a member-wise copy (shallow copy) of the class data.

Q54. If Person is a base class and Student is a derived class, which assignment is valid?

A. Person * p = new Student();

B. Student * s = new Person();

C. Student s = new Person();

D. Person p = new Student;

Difficulty: Medium | Importance: 5/5

Explanation:
A base class pointer can point to a derived class object (Upcasting), but a derived pointer cannot point to a base object without casting.

Q55. What happens to the pointer in a shallow copy when the original object is destroyed?

A. It becomes NULL

B. It becomes a dangling pointer

C. It is automatically reallocated

D. It remains valid

Difficulty: Medium | Importance: 4/5

Explanation:
In a shallow copy, if the original object deletes the memory, the copied object's pointer points to invalid memory (dangling pointer).

Q56. The const keyword in const Person & rhs ensures that:

A. The function is constant

B. The object rhs cannot be modified

C. The return type is constant

D. The pointer is null

Difficulty: Easy | Importance: 3/5

Explanation:
Passing by const reference ensures efficiency (no copy) and safety (the source object cannot be changed).

Q57. Can a protected member function be accessed directly in main() via an object?

A. Yes

B. No

C. Only if static

D. Only if virtual

Difficulty: Easy | Importance: 4/5

Explanation:
Protected members are not accessible from outside the class (like in main), only from within the class or derived classes.

Q58. In C++, Student IS-A Person implies that Student has:

A. Only its own properties

B. Only Person's properties

C. Person's properties plus its own

D. None of the above

Difficulty: Easy | Importance: 3/5

Explanation:
Inheritance implies the derived class inherits the base class's structure and behavior and can extend it.

Q59. To avoid memory errors with dynamic memory allocation in classes, one should implement:

A. Shallow Copy

B. Deep Copy

C. Virtual Destructor

D. Static Constructor

Difficulty: Medium | Importance: 5/5

Explanation:
Deep copy allocates new memory for the copy, ensuring two objects do not share the same dynamic memory block.

Q60. In C++, which casting operator is specifically designed to handle polymorphic behavior (checking types at runtime)?

A. static_cast

B. dynamic_cast

C. reinterpret_cast

D. const_cast

Difficulty: Medium | Importance: 5/5

Explanation:
Dynamic_cast is used for handling polymorphism and checks the inheritance hierarchy at runtime.

Q61. What happens if a user-defined derived class assignment operator does not explicitly call the base class assignment operator?

A. Base part is not assigned

B. Compiler calls it automatically

C. Runtime error

D. Syntax error

Difficulty: Hard | Importance: 5/5

Explanation:
In a user-defined assignment operator, the programmer must explicitly call the base class assignment operator; otherwise, the base part remains unchanged.

Q62. To override a function in a derived class, the function signature must match the base class function in:

A. Name only

B. Return type only

C. Name, parameters, and return type

D. Name and parameters only

Difficulty: Easy | Importance: 5/5

Explanation:
Function overriding requires the exact same signature (prototype) including name, parameters, and return type.

Q63. Which syntax is correct for explicitly calling the base class assignment operator from the derived class?

A. Base::operator=(rhs);

B. Base = rhs;

C. operator=(Base);

D. super.operator=(rhs);

Difficulty: Medium | Importance: 4/5

Explanation:
The scope resolution operator is used to explicitly call the base class version of the assignment operator.

Q64. Function Overloading occurs within the scope of:

A. Two different classes

B. One class

C. Global scope only

D. Namespace only

Difficulty: Easy | Importance: 4/5

Explanation:
Function overloading is done within the scope of a single class, whereas overriding involves inheritance.

Q65. What is 'Upcasting' in the context of inheritance?

A. Converting Base to Derived

B. Converting Derived to Base

C. Converting int to float

D. Converting pointer to reference

Difficulty: Medium | Importance: 4/5

Explanation:
Upcasting refers to converting a derived class reference or pointer to a base class reference or pointer.

Q66. Which casting operator allows reinterpreting the bit pattern of an operand (e.g., pointer to integer)?

A. static_cast

B. dynamic_cast

C. reinterpret_cast

D. const_cast

Difficulty: Hard | Importance: 3/5

Explanation:
Reinterpret_cast is used for low-level casting that reinterprets the bits of the expression.

Q67. If a derived class copy constructor does not specify a base class constructor in its initialization list, which base constructor is called?

A. Copy constructor

B. Default constructor

C. Parameterized constructor

D. None

Difficulty: Medium | Importance: 5/5

Explanation:
The compiler implicitly calls the base class default constructor if no specific base constructor is invoked in the initialization list.

Q68. In C++, the type used to declare a reference or pointer is known as its:

A. Dynamic type

B. Static type

C. Virtual type

D. Abstract type

Difficulty: Easy | Importance: 3/5

Explanation:
The static type of an identifier is the type used specifically for its declaration in the code.

Q69. To perform a 'Deep Copy' in a derived class constructor, what must be done regarding the base class?

A. Call base default constructor

B. Call base copy constructor

C. Use static_cast

D. Use memcpy

Difficulty: Medium | Importance: 5/5

Explanation:
The derived class should explicitly call the base class copy constructor in the initializer list to ensure the base part is also copied correctly.

Q70. Which of the following is NOT one of the four standard C++ casting operators?

A. static_cast

B. dynamic_cast

C. future_cast

D. const_cast

Difficulty: Easy | Importance: 3/5

Explanation:
The four standard casts are static_cast, dynamic_cast, const_cast, and reinterpret_cast.

Q71. Declaring two functions with the exact same prototype in the same class results in:

A. Overloading

B. Overriding

C. Duplicate declaration error

D. Runtime polymorphism

Difficulty: Medium | Importance: 4/5

Explanation:
Overriding only happens in inheritance; within the same class, identical signatures cause a compilation error.

Q72. What type of relationship does public inheritance model in C++?

A. HAS-A

B. IS-A

C. USES-A

D. OWNS-A

Difficulty: Easy | Importance: 5/5

Explanation:
Public inheritance represents an IS-A relationship (e.g., Student IS A Person).

Q73. Downcasting refers to casting from:

A. Derived to Base

B. Base to Derived

C. int to char

D. void to int

Difficulty: Medium | Importance: 4/5

Explanation:
Downcasting is the process of casting a base class pointer/reference to a derived class pointer/reference.

Q74. Why might dynamic_cast fail (return NULL) during downcasting?

A. If the object is not actually of the derived type

B. If the cast is implicit

C. If the system is out of memory

D. If the base class is abstract

Difficulty: Hard | Importance: 5/5

Explanation:
Dynamic_cast performs a runtime check and returns NULL (for pointers) if the object is not actually an instance of the target derived class.

Q75. C++ is described as which type of language regarding type usage?

A. Loosely typed

B. Strongly typed

C. Untyped

D. Dynamic only

Difficulty: Easy | Importance: 3/5

Explanation:
C++ is strongly typed, meaning types must be converted (cast) explicitly when they are not compatible.

Q76. In the context of the assignment operator, why is a check for self-assignment (e.g., if(this == &rhs)) important?

A. To save memory

B. To prevent deleting data before copying

C. To increase speed

D. To call the destructor

Difficulty: Medium | Importance: 4/5

Explanation:
Although implied in deep copy logic, failing to check for self-assignment can lead to deleting the object's own data before copying it back.

Q77. When Student inherits from Person, can a Student object reference be assigned to a Person reference?

A. Yes, implicitly

B. No, never

C. Only with reinterpret_cast

D. Only if private inheritance

Difficulty: Medium | Importance: 5/5

Explanation:
Because a Student IS-A Person, a derived reference can be implicitly assigned to a base reference.

Q78. In function overriding, if a derived class function calls the base class function with the same name without using the scope resolution operator, what is the result?

A. Successful call

B. Compilation error

C. Infinite recursion

D. Linker error

Difficulty: Medium | Importance: 5/5

Explanation:
Calling the function without :: causes the derived function to call itself repeatedly, leading to stack overflow.

Q79. Which type of inheritance models the 'Implemented in terms of' relationship?

A. Public Inheritance

B. Private Inheritance

C. Protected Inheritance

D. Virtual Inheritance

Difficulty: Medium | Importance: 5/5

Explanation:
Private inheritance is used when we want to reuse code (implementation) without exposing the interface (IS-A relationship).

Q80. In Public Inheritance, public members of the base class become _____ members of the derived class.

A. Public

B. Private

C. Protected

D. Hidden

Difficulty: Easy | Importance: 5/5

Explanation:
In public inheritance, the access level of public members remains public in the derived class.

FREE ALL SUBJECTS HANDOUTS, MCQS FILES, AND QUIZ TEST POWERED BY VUEDU

VUEDU

Q81. In Private Inheritance, public and protected members of the base class become _____ members of the derived class.

A. Public

B. Protected

C. Private

D. Hidden

Difficulty: Medium | Importance: 5/5

Explanation:
Private inheritance restricts access, making both public and protected base members private in the derived class.

Q82. If no access specifier is mentioned when deriving a class (e.g., class Child : Parent), what is the default inheritance type?

A. Public

B. Private

C. Protected

D. Static

Difficulty: Easy | Importance: 4/5

Explanation:
For classes, the default inheritance type is private if not explicitly specified.

Q83. Can a derived class constructor explicitly call the constructor of its Indirect Base Class in its initialization list?

A. Yes, always

B. No, never

C. Only if virtual

D. Only if public

Difficulty: Medium | Importance: 5/5

Explanation:
A child class can only initialize its Direct Base Class; it cannot skip a level to initialize an Indirect Base Class.

Q84. What is a 'Direct Base Class'?

A. The class at the top of hierarchy

B. A class explicitly listed in the derived class header

C. A friend class

D. A nested class

Difficulty: Easy | Importance: 3/5

Explanation:
A direct base class is the one immediately inherited from, appearing after the colon in the class definition.

Q85. In the context of the 'Set' and 'Collection' example, why was Private Inheritance used?

A. To allow duplicates

B. To restrict the interface (Specialization)

C. To allow public access

D. To save memory

Difficulty: Medium | Importance: 4/5

Explanation:
Private inheritance was used to hide methods like AddElement that allow duplicates, enforcing the Set's unique constraint.

Q86. What happens to private members of a base class when inherited publicly?

A. Become Public

B. Become Private

C. Become Protected

D. Remain Hidden (Inaccessible)

Difficulty: Easy | Importance: 5/5

Explanation:
Private members of a base class are never accessible directly in the derived class, regardless of inheritance type.

Q87. To call the base class version of a function Print() from within the derived class overriding function, which syntax is correct?

A. Print()

B. Base.Print()

C. Base::Print()

D. super::Print()

Difficulty: Easy | Importance: 5/5

Explanation:
The scope resolution operator (BaseClassName::FunctionName) is required to explicitly call the base implementation.

Q88. In Protected Inheritance, public members of the base class become _____ members of the derived class.

A. Public

B. Private

C. Protected

D. Hidden

Difficulty: Medium | Importance: 4/5

Explanation:
Protected inheritance demotes public members of the base class to protected status in the derived class.

Q89. 'Specialization' in inheritance implies that the derived class is:

A. Behaviorally compatible

B. Behaviorally incompatible

C. Identical to base

D. Abstract

Difficulty: Hard | Importance: 4/5

Explanation:
Specialization means the derived class restricts behavior, so it cannot always substitute the base class (incompatible IS-A).

Q90. Which type of inheritance is behaviorally compatible (IS-A relationship)?

A. Public

B. Private

C. Protected

D. Restricted

Difficulty: Easy | Importance: 5/5

Explanation:
Public inheritance models the IS-A relationship where the derived object can substitute the base object.

Q91. In the 'Collection' vs 'Set' example, Set allows:

A. Duplicate elements

B. No duplicate elements

C. Sorted elements

D. Negative numbers only

Difficulty: Easy | Importance: 3/5

Explanation:
The Set class was specialized to enforce uniqueness, unlike the generic Collection class.

Q92. What is the correct syntax to inherit Parent privately in class Child?

A. class Child : public Parent

B. class Child : protected Parent

C. class Child : private Parent

D. class Child :: Parent

Difficulty: Easy | Importance: 4/5

Explanation:
The keyword private before the base class name specifies private inheritance.

Q93. If a function is overridden in a derived class, it must have:

A. Same name only

B. Same return type only

C. Same name, parameters, and return type

D. Different parameters

Difficulty: Medium | Importance: 5/5

Explanation:
Overriding requires the exact same signature (prototype) as the base class function.

Q94. In Private Inheritance, can the object of the derived class access public member functions of the base class directly in main()?

A. Yes

B. No

C. Only if static

D. Only const functions

Difficulty: Medium | Importance: 5/5

Explanation:
Since public members become private in the derived class, they are inaccessible from outside (e.g., in main).

Q95. When Child overrides GrandParent::Print, calling obj.GrandParent::Print() in main will:

A. Error

B. Call Child::Print

C. Call GrandParent::Print

D. Call Parent::Print

Difficulty: Medium | Importance: 4/5

Explanation:
Explicitly using the scope resolution operator allows bypassing the override and calling the specific base version.

Q96. In Private Inheritance, who can convert a pointer of the derived class to a pointer of the base class?

A. Anyone

B. Only member functions and friends of the derived class

C. Derived classes of the derived class

D. Global functions

Difficulty: Medium | Importance: 5/5

Explanation:
In private inheritance, the conversion from Derived* to Base* is accessible only to the derived class's own members and friends.

Q97. Which relationship is modeled by Private and Protected Inheritance?

A. IS-A

B. HAS-A

C. Implemented in terms of

D. Uses-A

Difficulty: Medium | Importance: 5/5

Explanation:
Private and Protected inheritance are used to model the 'Implemented in terms of' relationship, often for code reuse or specialization.

Q98. In Protected Inheritance, public members of the base class become _____ members of the derived class.

A. Public

B. Private

C. Protected

D. Hidden

Difficulty: Easy | Importance: 4/5

Explanation:
When inheriting protectively, public and protected members of the base class become protected members of the derived class.

Q99. If a class is derived privately from a base class, can a class derived from this child class access the grandparent's members?

A. Yes, always

B. No, never

C. Only public members

D. Only protected members

Difficulty: Medium | Importance: 4/5

Explanation:
In private inheritance, all members of the base class become private in the derived class, making them inaccessible to further derived classes (grandchildren).

Q100. What is the primary drawback of using switch/if-else logic to handle different object types instead of polymorphism?

A. Compilation error

B. Delocalized Code

C. Memory leak

D. Faster execution

Difficulty: Medium | Importance: 5/5

Explanation:
Using switch statements leads to delocalized code, requiring logic duplication whenever a new operation or shape is added.

Q101. In the context of the 'Shape' example, without virtual functions, calling draw() on a Shape* array containing Line objects results in:

A. Calling Line::draw()

B. Calling Shape::draw()

C. Runtime Error

D. Compilation Error

Difficulty: Medium | Importance: 5/5

Explanation:
Without virtual functions, the compiler uses static binding, invoking the base class (Shape) method instead of the derived class method.

Q102. When an object of a derived class is created using private inheritance, which constructor is called first?

A. Derived Constructor

B. Base Constructor

C. Both simultaneously

D. None

Difficulty: Easy | Importance: 5/5

Explanation:
Regardless of inheritance type, the base class constructor is always executed before the derived class constructor.

Q103. Which casting operator is used in the text to cast Shape* to Line* in the switch-case solution?

A. dynamic_cast

B. static_cast

C. reinterpret_cast

D. const_cast

Difficulty: Hard | Importance: 3/5

Explanation:
The text demonstrates using static_cast to convert the base pointer to a derived pointer within the switch logic.

Q104. Specialization (Restriction) involves deriving a class that is behaviorally _____ with the base class.

A. Compatible

B. Incompatible

C. Identical

D. Parallel

Difficulty: Medium | Importance: 4/5

Explanation:
Specialization implies the derived class restricts behavior (e.g., age range), making it behaviorally incompatible (cannot always replace base).

Q105. If class Child : protected Parent, who can convert Child* to Parent*?

A. Everyone

B. Only Child

C. Child, its friends, and classes derived from Child

D. No one

Difficulty: Hard | Importance: 4/5

Explanation:
In protected inheritance, conversion is allowed for the derived class, its friends, and any classes further derived from it.

Q106. Private data members of a base class are accessible in a derived class:

A. Directly

B. Via public methods only

C. Never

D. Only if static

Difficulty: Easy | Importance: 5/5

Explanation:
Private members of a base class are never directly accessible in a derived class, regardless of the inheritance type.

Q107. Which type of inheritance lies between Public and Private inheritance in terms of access restriction?

A. Static

B. Protected

C. Virtual

D. Friend

Difficulty: Easy | Importance: 3/5

Explanation:
Protected inheritance is intermediate; it allows derived classes access but restricts the outside world, unlike private (restricts derived's children) or public (open).

Q108. In the 'Adult' vs 'Person' example, Adult restricts the age range to:

A. 0-125

B. 18-125

C. 21-100

D. No restriction

Difficulty: Easy | Importance: 3/5

Explanation:
The Adult class specializes Person by restricting the valid age range to 18-125.

Q109. If a function drawShapes takes Shape* [], what is the static type of the array elements?

A. Line*

B. Circle*

C. Shape*

D. void*

Difficulty: Easy | Importance: 4/5

Explanation:
The array is declared as Shape*, so the static type of its elements is a pointer to Shape.

Q110. In Private Inheritance, public members of the base class become _____ in the derived class.

A. Public

B. Protected

C. Private

D. Deleted

Difficulty: Medium | Importance: 5/5

Explanation:
Private inheritance makes all inherited members (public and protected) private in the derived class.

Q111. What feature is used to solve the problem of calling the correct draw() function for different shapes automatically?

A. Friend Functions

B. Virtual Functions

C. Inline Functions

D. Static Functions

Difficulty: Medium | Importance: 5/5

Explanation:
Virtual functions allow dynamic binding, ensuring the correct derived class function is called at runtime.

Q112. Can a user-defined copy constructor in a privately derived class call the base class copy constructor?

A. Yes

B. No

C. Only if public

D. Only if virtual

Difficulty: Medium | Importance: 4/5

Explanation:
Yes, the derived class constructor (even in private inheritance) can and should call the base class copy constructor in its initialization list.

Q113. Which relationship allows a base class pointer to point to a derived class object?

A. IS-A

B. HAS-A

C. USES-A

D. KNOWS-A

Difficulty: Easy | Importance: 5/5

Explanation:
The IS-A relationship (Public Inheritance) guarantees that a derived object is a base object, allowing pointer substitution.

Q114. In the context of inheritance, an 'Anonymous Object' refers to:

A. A unnamed temporary variable

B. The base class part within a derived object

C. A static object

D. A null pointer

Difficulty: Medium | Importance: 3/5

Explanation:
Every derived class object contains an anonymous instance of its base class.

Q115. If B is a private base of D, can main() function access public members of B through an object of D?

A. Yes

B. No

C. Only if static

D. Sometimes

Difficulty: Easy | Importance: 5/5

Explanation:
Since inheritance is private, B's public members become private in D and are inaccessible to the outside world (main).

Q116. What is the main drawback of using switch statements to handle different object types instead of virtual functions?

A. Slower compilation

B. Delocalized code

C. Runtime errors

D. Memory leaks

Difficulty: Medium | Importance: 5/5

Explanation:
Using switch statements leads to code being scattered (delocalized) across different functions, making maintenance difficult.

Q117. In C++, which keyword is used to implement dynamic binding?

A. static

B. virtual

C. friend

D. inline

Difficulty: Easy | Importance: 5/5

Explanation:
The virtual keyword tells the compiler to perform dynamic binding (resolve function calls at runtime).

Q118. What is a Pure Virtual Function?

A. A function with empty body {}

B. A function initialized to 0

C. A static function

D. A function without return type

Difficulty: Easy | Importance: 5/5

Explanation:
A pure virtual function is declared with = 0 at the end of its header (e.g., virtual void draw() = 0;).

Q119. A class that contains at least one pure virtual function is known as:

A. Concrete Class

B. Abstract Class

C. Static Class

D. Friend Class

Difficulty: Easy | Importance: 5/5

Explanation:
If a class has one or more pure virtual functions, it becomes an Abstract Class and cannot be instantiated.

Q120. If a base class destructor is NOT virtual, what happens when deleting a derived class object via a base class pointer?

A. Compilation error

B. Only base destructor is called

C. Only derived destructor is called

D. Both destructors are called

Difficulty: Medium | Importance: 5/5

Explanation:
Without a virtual destructor, only the base class destructor executes, potentially causing memory leaks (derived part not cleaned up).

Q121. Static binding means the target function is selected at:

A. Runtime

B. Compile time

C. Link time

D. Load time

Difficulty: Easy | Importance: 4/5

Explanation:
Static binding (early binding) occurs when the compiler determines which function to call during compilation.

Q122. Dynamic binding means the target function is selected at:

A. Runtime

B. Compile time

C. Link time

D. Load time

Difficulty: Easy | Importance: 4/5

Explanation:
Dynamic binding (late binding) occurs when the program determines which function to call while it is running.

Q123. Can an Abstract Class be instantiated (can you create an object of it)?

A. Yes, always

B. No, never

C. Only if static

D. Only via pointers

Difficulty: Easy | Importance: 5/5

Explanation:
Abstract classes exist only to be inherited from; you cannot create direct instances (objects) of them.

Q124. What is the output of delete pShape; if pShape points to a Rectangle but ~Shape() is NOT virtual?

A. Shape destructor called

B. Rectangle destructor called

C. Both called

D. Runtime Error

Difficulty: Medium | Importance: 5/5

Explanation:
Since the pointer is of type Shape* and the destructor is not virtual, only the Shape destructor executes.

Q125. To prevent memory leaks when using polymorphism, the base class destructor should be:

A. Private

B. Static

C. Virtual

D. Pure

Difficulty: Medium | Importance: 5/5

Explanation:
Making the base destructor virtual ensures the derived class destructor is called first, properly cleaning up resources.

Q126. If a derived class does not override a pure virtual function from its abstract base class, the derived class becomes:

A. Concrete

B. Abstract

C. Static

D. Invalid

Difficulty: Medium | Importance: 4/5

Explanation:
A derived class remains abstract until it provides implementations for ALL inherited pure virtual functions.

Q127. Which type of class implements a concrete concept and can be instantiated?

A. Abstract Class

B. Concrete Class

C. Virtual Class

D. Pure Class

Difficulty: Easy | Importance: 3/5

Explanation:
Concrete classes have no pure virtual functions and can be used to create objects.

Q128. In the context of the Shape hierarchy, Shape::draw() is best modeled as a:

A. Static function

B. Friend function

C. Pure virtual function

D. Inline function

Difficulty: Medium | Importance: 4/5

Explanation:
Since a generic 'Shape' cannot be drawn, the draw function should be pure virtual, forcing derived classes to implement it.

Q129. Polymorphism allows a sender to send a message without knowing the exact _____ of the receiver.

A. Address

B. Class

C. Size

D. Name

Difficulty: Medium | Importance: 3/5

Explanation:
Polymorphism allows treating different objects uniformly (e.g., as Shape*) without knowing if they are Lines, Circles, etc.

Q130. What is the correct syntax to declare a pure virtual function?

A. virtual void func() {}

B. virtual void func() = NULL;

C. virtual void func() = 0;

D. void func() = 0;

Difficulty: Easy | Importance: 5/5

Explanation:
The = 0 syntax specifically designates a virtual function as 'pure'.

Q131. When delete is called on a base pointer with a virtual destructor, the order of destruction is:

A. Base then Derived

B. Derived then Base

C. Random

D. Simultaneous

Difficulty: Medium | Importance: 4/5

Explanation:
Virtual destructors ensure the Derived destructor runs first to clean up its part, followed by the Base destructor.

Q132. Using switch logic to handle types violates which principle of maintainable code?

A. Encapsulation

B. Localization

C. Inheritance

D. Polymorphism

Difficulty: Hard | Importance: 3/5

Explanation:
It creates 'delocalized' code, where logic for a single concept is scattered across multiple functions/files.

Q133. Abstract classes are typically found at which level of the class hierarchy tree?

A. Leaves

B. Root or near root

C. Middle only

D. Separate branch

Difficulty: Medium | Importance: 3/5

Explanation:
Abstract classes usually define common interfaces at the top (root) of the hierarchy, while concrete classes are at the bottom (leaves).

Q134. Which of the following creates a 'memory leak' in the context of inheritance?

A. Virtual Destructor

B. Non-virtual Destructor

C. Pure Virtual Function

D. Static Binding

Difficulty: Medium | Importance: 5/5

Explanation:
Deleting a derived object through a base pointer without a virtual destructor fails to free the derived part's memory.

Q135. In C++, a function declared as virtual void draw() = 0; is known as a:

A. Static function

B. Friend function

C. Pure virtual function

D. Inline function

Difficulty: Easy | Importance: 5/5

Explanation:
The = 0 syntax specifically designates a virtual function as a pure virtual function, meaning it has no implementation in the base class.

Q136. A class that contains at least one pure virtual function is called:

A. Concrete Class

B. Abstract Class

C. Virtual Class

D. Static Class

Difficulty: Easy | Importance: 5/5

Explanation:
A class with one or more pure virtual functions is abstract and cannot be instantiated.

Q137. What is the primary purpose of a Virtual Destructor?

A. To save memory

B. To call the derived class destructor

C. To call the base class constructor

D. To create a V-Table

Difficulty: Medium | Importance: 5/5

Explanation:
When deleting a derived object via a base pointer, a virtual destructor ensures the derived class destructor is called to prevent memory leaks.

Q138. What data structure does the compiler build for each class having virtual functions?

A. Stack

B. Queue

C. V-Table

D. Linked List

Difficulty: Medium | Importance: 4/5

Explanation:
The compiler creates a Virtual Function Table (V-Table) containing pointers to the virtual functions for that class.

Q139. Dynamic binding (or late binding) determines the target function call at:

A. Compile time

B. Link time

C. Run time

D. Load time

Difficulty: Easy | Importance: 5/5

Explanation:
Dynamic binding resolves the function call during program execution based on the actual object type.

Q140. Why is treating an array of derived objects as an array of base objects polymorphically dangerous?

A. Compilation error

B. Syntax error

C. Pointer arithmetic error

D. Memory overflow

Difficulty: Hard | Importance: 5/5

Explanation:
The size of derived objects differs from base objects, causing the compiler to calculate incorrect addresses when iterating, leading to runtime errors.

Q141. A class that provides implementations for all its pure virtual functions and can be instantiated is called a:

A. Abstract Class

B. Concrete Class

C. Base Class

D. Interface

Difficulty: Easy | Importance: 3/5

Explanation:
A concrete class has no remaining pure virtual functions and can be used to create objects.

Q142. What is the overhead associated with using virtual functions?

A. None

B. Only compilation time

C. Memory for V-Table and extra pointer lookup

D. Disk space

Difficulty: Medium | Importance: 4/5

Explanation:
Virtual functions incur overhead due to the storage of the V-Table and the extra processing required to look up function addresses.

Q143. In the context of inheritance, virtual void draw() (without = 0) implies inheriting:

A. Interface only

B. Implementation only

C. Interface and Implementation

D. None of the above

Difficulty: Medium | Importance: 4/5

Explanation:
A simple virtual function provides a default implementation that derived classes inherit, along with the interface.

Q144. If a derived class does not override a pure virtual function inherited from its base class, the derived class becomes:

A. Concrete

B. Abstract

C. Static

D. Invalid

Difficulty: Medium | Importance: 5/5

Explanation:
The derived class remains abstract and cannot be instantiated until all inherited pure virtual functions are implemented.

Q145. Polymorphism in C++ works correctly with:

A. Objects passed by value

B. Arrays of objects

C. Pointers and References

D. Static variables

Difficulty: Easy | Importance: 5/5

Explanation:
Polymorphism requires pointers or references to base classes to support dynamic binding to derived objects.

Q146. In the V-Table, what does each entry correspond to?

A. Data member

B. Virtual function pointer

C. Static variable

D. Constructor

Difficulty: Medium | Importance: 4/5

Explanation:
The V-Table contains pointers to the code of the virtual functions for the class.

Q147. Static binding means the target function is selected at:

A. Run time

B. Compile time

C. Link time

D. Loading time

Difficulty: Easy | Importance: 4/5

Explanation:
Static binding occurs when the compiler determines the function call based on the static type of the pointer/object.

Q148. When delete is used on a base class pointer with a virtual destructor, which destructor is called first?

A. Base Class Destructor

B. Derived Class Destructor

C. Both simultaneously

D. Random order

Difficulty: Medium | Importance: 5/5

Explanation:
The derived class destructor is called first to clean up specific resources, followed by the base class destructor.

Q149. In the Payroll case study, the Employee class is an example of:

A. Concrete Class

B. Abstract Base Class

C. Derived Class

D. Helper Class

Difficulty: Medium | Importance: 3/5

Explanation:
Employee declares a pure virtual function calcSalary, making it an abstract base class.

Q150. Can you create an instance (object) of an Abstract Class?

A. Yes

B. No

C. Only if static

D. Only if const

Difficulty: Easy | Importance: 5/5

Explanation:
Abstract classes are incomplete (have undefined pure virtual functions) and cannot be instantiated directly.

Q151. The sizeof(Derived) is typically _____ sizeof(Base).

A. Equal to

B. Less than

C. Greater than

D. Unrelated to

Difficulty: Medium | Importance: 3/5

Explanation:
Derived classes usually contain their own data members in addition to the base class members, making them larger.

Q152. What happens if you declare a function as virtual in the base class and override it in the derived class without the virtual keyword?

A. It becomes non-virtual

B. It remains virtual

C. Compiler error

D. Runtime error

Difficulty: Medium | Importance: 4/5

Explanation:
Once a function is declared virtual in the base class, it remains virtual throughout the inheritance hierarchy automatically.

Q153. To inherit only the interface (prototype) without implementation, one should use:

A. Inline functions

B. Friend functions

C. Pure Virtual functions

D. Static functions

Difficulty: Medium | Importance: 4/5

Explanation:
Pure virtual functions force derived classes to provide their own implementation, effectively inheriting only the interface.

Q154. What is the 'Moral' mentioned regarding virtual functions in the text?

A. Make everything virtual

B. Think about performance before making functions virtual

C. Avoid virtual functions completely

D. Use virtual functions for arrays

Difficulty: Easy | Importance: 3/5

Explanation:
Due to the associated overhead, virtual functions should be used judiciously based on performance requirements.

Q155. Why should you avoid using arrays of objects polymorphically in C++?

A. Compilation error

B. Memory leak

C. Incorrect pointer arithmetic

D. Virtual table corruption

Difficulty: Hard | Importance: 5/5

Explanation:
When iterating an array of derived objects treated as base objects, the compiler calculates addresses using the base class size, leading to incorrect locations.

Q156. In C++, what is the correct syntax for a class Phone inheriting from both Transmitter and Receiver?

A. class Phone : public Transmitter, Receiver

B. class Phone : public Transmitter, public Receiver

C. class Phone : public Transmitter : public Receiver

D. class Phone inherits Transmitter, Receiver

Difficulty: Easy | Importance: 5/5

Explanation:
In multiple inheritance, each base class must be separated by a comma and can have its own access specifier.

Q157. If two base classes have a function with the same signature, calling it from a derived class object results in:

A. Runtime Error

B. Ambiguity Error

C. Calling the first base's function

D. Calling the last base's function

Difficulty: Medium | Importance: 5/5

Explanation:
The compiler cannot decide which base class's function to call, resulting in an ambiguity error.

Q158. How can you resolve the ambiguity when two base classes possess a function with the same name?

A. Using virtual keyword

B. Using scope resolution operator

C. Overriding the function

D. Using static_cast

Difficulty: Medium | Importance: 4/5

Explanation:
You must explicitly specify the class name using the scope resolution operator (e.g., obj.Base1::function()).

Q159. The 'Diamond Problem' in inheritance occurs when:

A. A class inherits from multiple classes

B. Two classes inherit from a single class

C. A class inherits from two classes that share a common base

D. A class has a private base class

Difficulty: Medium | Importance: 5/5

Explanation:
This creates two copies of the common base class in the final derived class, causing ambiguity and memory redundancy.

Q160. What is the specific solution in C++ to avoid the Diamond Problem (multiple copies of a base class)?

A. Multiple Inheritance

B. Virtual Inheritance

C. Abstract Classes

D. Friend Functions

Difficulty: Easy | Importance: 5/5

Explanation:
Virtual inheritance ensures that only one copy of the base class is generated in the derived class hierarchy.

FREE ALL SUBJECTS HANDOUTS, MCQS FILES, AND QUIZ TEST POWERED BY VUEDU

VUEDU QUIZ TEST

Q161. When LandVehicle and WaterVehicle both inherit from Vehicle, and AmphibiousVehicle inherits from both, how many copies of Vehicle exist in AmphibiousVehicle by default?

A. One

B. Two

C. Zero

D. Three

Difficulty: Medium | Importance: 4/5

Explanation:
Without virtual inheritance, each intermediate base class holds its own copy of the grandparent class.

Q162. Which keyword is used to implement virtual inheritance?

A. static

B. virtual

C. friend

D. volatile

Difficulty: Easy | Importance: 5/5

Explanation:
The keyword virtual is placed before the base class name in the inheritance declaration (e.g., class Derived : public virtual Base).

Q163. In the context of polymorphic arrays, if sizeof(Shape) is 10 and sizeof(Line) is 15, _shape[1] will point to address _____ if _shape is actually a Line array.

A. Base address + 10

B. Base address + 15

C. Base address + 4

D. Base address + 20

Difficulty: Hard | Importance: 4/5

Explanation:
The compiler uses the static type (Shape) size to calculate the offset, which is 10, but the actual object is 15 bytes away, causing an error.

Q164. What is the primary motivation for Generic Programming?

A. To use virtual functions

B. To avoid code duplication for different data types

C. To increase compilation speed

D. To hide implementation details

Difficulty: Easy | Importance: 4/5

Explanation:
Generic programming allows writing a single function or class that works with multiple data types, avoiding repetitive code.

Q165. In a multiple inheritance scenario, if Phone inherits from Transmitter, can a Transmitter* point to a Phone object?

A. Yes

B. No

C. Only with reinterpret_cast

D. Only if inheritance is private

Difficulty: Easy | Importance: 4/5

Explanation:
A base class pointer can always point to a derived class object in public inheritance.

Q166. If AmphibiousVehicle virtually inherits from Vehicle (via intermediate classes), how many copies of weight (a member of Vehicle) will it have?

A. One

B. Two

C. Three

D. Zero

Difficulty: Medium | Importance: 5/5

Explanation:
Virtual inheritance ensures exactly one copy of the anonymous base class object exists in the hierarchy.

Q167. What happens if you try to access a member of a virtual base class through a derived class that inherits it through multiple paths?

A. Ambiguity error

B. Runtime error

C. Successful access

D. Compiler warning

Difficulty: Medium | Importance: 4/5

Explanation:
Since there is only one shared copy of the virtual base class, there is no ambiguity in accessing its members.

Q168. If a pointer to a base class Transmitter is used to call a function, which function is called if it is NOT virtual?

A. Phone's function

B. Transmitter's function

C. Receiver's function

D. None

Difficulty: Easy | Importance: 4/5

Explanation:
For non-virtual functions, the compiler uses static binding based on the pointer type (Transmitter).

Q169. In multiple inheritance class C : public A, public B, the object of class C contains:

A. Only A's members

B. Only B's members

C. Members of both A and B

D. Only C's own members

Difficulty: Easy | Importance: 3/5

Explanation:
The derived class inherits all data members and functions from all its base classes.

Q170. Using an array of pointers Shape* [] works correctly for polymorphism because:

A. Pointers are always the same size

B. Pointers are virtual

C. Pointers are dynamic

D. Shape is abstract

Difficulty: Medium | Importance: 5/5

Explanation:
All pointers (e.g., on 32-bit systems) are typically 4 bytes, so pointer arithmetic calculates the correct address for the next pointer.

Q171. When is virtual inheritance NOT recommended?

A. When dealing with diamond problem

B. When distinct copies of base members are needed

C. When using abstract classes

D. When using polymorphism

Difficulty: Medium | Importance: 3/5

Explanation:
If you explicitly need separate copies of the base class data for each intermediate class, do not use virtual inheritance.

Q172. In the text, the function printArray was overloaded to demonstrate the need for:

A. Inheritance

B. Polymorphism

C. Templates (Generic Programming)

D. Encapsulation

Difficulty: Easy | Importance: 3/5

Explanation:
Writing the same function logic for int, char, and double illustrates the redundancy that generic programming solves.

Q173. Can a derived class inherit from both a public base class and a private base class simultaneously?

A. Yes

B. No

C. Only if virtual

D. Only if friend

Difficulty: Easy | Importance: 3/5

Explanation:
C++ allows mixed access specifiers for multiple base classes (e.g., class D : public A, private B).

Q174. Which type of inheritance is represented by the relationship 'AmphibiousVehicle is a LandVehicle AND a WaterVehicle'?

A. Single Inheritance

B. Multilevel Inheritance

C. Multiple Inheritance

D. Hierarchical Inheritance

Difficulty: Easy | Importance: 5/5

Explanation:
Multiple inheritance allows a class to combine the features of multiple direct base classes.

Q175. In C++, which keyword is used interchangeably with 'class' in a template declaration?

A. type

B. typename

C. object

D. template_id

Difficulty: Easy | Importance: 5/5

Explanation:
In template declarations, 'typename' and 'class' can be used interchangeably to define type parameters.

Q176. What is the primary goal of Generic Programming?

A. To depend on data structures

B. To be independent of data structures

C. To use only integer arrays

D. To increase code size

Difficulty: Medium | Importance: 5/5

Explanation:
Generic programming aims to write code that is independent of specific data structures or types.

Q177. In the generic 'find' function discussed, what are the two required operations for the container?

A. Increment (++) and Dereference ()

B. Addition (+) and Subtraction (-)

C. Input (>>) and Output (<<)

D. New and Delete

Difficulty: Medium | Importance: 4/5

Explanation:
The generic algorithm requires traversing the container (++) and accessing elements () to compare them.

Q178. Which syntax is correct for defining a class template member function outside the class?

A. RetType ClassName::Func()

B. template<class T> RetType ClassName<T>::Func()

C. template<T> RetType ClassName::Func()

D. RetType ClassName<T>::Func()

Difficulty: Hard | Importance: 5/5

Explanation:
Member functions defined outside a class template must be preceded by the template declaration and scope resolution with <T>.

Q179. In the 'Complex' class example, why is a member template copy constructor required?

A. To copy same types

B. To allow assignment between different types

C. To delete objects

D. To overload operators

Difficulty: Hard | Importance: 4/5

Explanation:
A member template copy constructor allows copying a Complex<float> object to a Complex<double> object.

Q180. What does the 'find' function return if the element is NOT found in the container?

A. NULL

B. The 'beyond' pointer

C. -1

D. false

Difficulty: Medium | Importance: 4/5

Explanation:
In the refined generic implementation, it returns the 'beyond' pointer (end of range) to indicate failure.

Q181. How can we set a default policy for a template parameter?

A. template< typename T = DefaultType >

B. template< typename T : DefaultType >

C. template< typename T == DefaultType >

D. template< typename T(DefaultType) >

Difficulty: Medium | Importance: 3/5

Explanation:
Default template arguments are specified using the assignment operator '=', similar to default function parameters.

Q182. In the Vector class template, what happens in the constructor if the size is 0?

A. ptr is set to new T[0]

B. ptr is set to 0 (NULL)

C. Program crashes

D. Size is reset to 10

Difficulty: Medium | Importance: 3/5

Explanation:
The implementation explicitly checks if size != 0; otherwise, it assigns 0 to the internal pointer.

Q183. Which concept allows a single class template to operate on different data types?

A. Polymorphism

B. Inheritance

C. Templates

D. Encapsulation

Difficulty: Easy | Importance: 5/5

Explanation:
Templates allow classes and functions to operate with generic types, facilitating code reuse.

Q184. What is the correct way to instantiate a Vector class template for integers?

A. Vector int obj;

B. Vector<int> obj;

C. Vector(int) obj;

D. int Vector obj;

Difficulty: Easy | Importance: 5/5

Explanation:
Class templates must be explicitly instantiated with the data type in angle brackets.

Q185. In the Vector class assignment operator, what is the first critical step?

A. Copy the data

B. Check for self-assignment

C. Delete old memory

D. Resize the array

Difficulty: Medium | Importance: 4/5

Explanation:
The operator checks 'if (this != &right)' to prevent errors during self-assignment.

Q186. What problem does the compiler avoid by generating only required template instances?

A. Code Bloat

B. Syntax Errors

C. Logic Errors

D. Runtime Errors

Difficulty: Medium | Importance: 4/5

Explanation:
The compiler only generates code for template instantiations that are actually used, reducing unnecessary code size.

Q187. In the 'compare' function example (Lecture 33), what does a return value of 0 indicate?

A. Strings are equal

B. First string is larger

C. Second string is larger

D. Error occurred

Difficulty: Easy | Importance: 3/5

Explanation:
Standard comparison logic returns 0 when the difference between characters is zero, meaning strings are equal.

Q188. Why is 'const' used in the function signature 'int getSize() const'?

A. To return a constant integer

B. To allow modification of data members

C. To prevent modification of the object

D. To make the function static

Difficulty: Easy | Importance: 4/5

Explanation:
A const member function guarantees it will not modify any non-mutable data members of the object.

Q189. In the context of the Vector class, what does the destructor do?

A. Deletes the size variable

B. Deletes the ptr array

C. Sets size to zero

D. Nothing

Difficulty: Medium | Importance: 3/5

Explanation:
The destructor calls 'delete [] ptr;' to free the dynamically allocated memory.

Q190. What happens if 'operator[]' is called with an invalid index in the Vector class?

A. Returns garbage value

B. Returns 0

C. Exits the program

D. Throws an exception

Difficulty: Medium | Importance: 3/5

Explanation:
The code checks bounds and calls 'exit(1)' if the index is out of range.

Q191. Member functions of a template class implicitly become what?

A. Virtual functions

B. Function templates

C. Static functions

D. Friend functions

Difficulty: Medium | Importance: 4/5

Explanation:
Since they rely on the class's template parameter, member functions automatically behave like function templates.

Q192. What does the 'beyond' pointer represent in the generic 'find' function?

A. The last element

B. One location past the last element

C. The first element

D. The middle element

Difficulty: Medium | Importance: 4/5

Explanation:
The 'beyond' pointer serves as a sentinel value indicating the end of the search range.

Q193. In the statement 'Complex<double> dc = fc;', where fc is Complex<float>, which constructor is called?

A. Default constructor

B. Standard copy constructor

C. Template copy constructor

D. Conversion operator

Difficulty: Hard | Importance: 5/5

Explanation:
The compiler matches the source type (float) to the template parameter (U) of the member template copy constructor.

Q194. In the context of template class specialization, what issue arises when copying a Vector<char*> object without a custom copy constructor?

A. Syntax Error

B. Shallow Copy

C. Deep Copy

D. Compilation Error

Difficulty: Medium | Importance: 5/5

Explanation:
The default copy constructor performs a bitwise copy, leading to shallow copy issues where two objects point to the same memory.

Q195. Which keyword sequence is used to define a complete specialization of a class template?

A. template <T>

B. template <>

C. template <class T>

D. specialization <>

Difficulty: Medium | Importance: 5/5

Explanation:
Complete specialization is indicated by template <> with no parameters in the angle brackets.

Q196. In the specialized Vector<char*> class, what is the type of the data member ptr?

A. char*

B. char

C. char[]

D. Vector*

Difficulty: Hard | Importance: 4/5

Explanation:
Since it stores an array of character pointers (strings), the pointer to the array must be a pointer-to-pointer (char**).

Q197. What is a 'non-type parameter' in a C++ template definition?

A. A parameter that represents a data type

B. A constant value known at compile time

C. A pointer to a function

D. A variable calculated at runtime

Difficulty: Medium | Importance: 5/5

Explanation:
Non-type parameters, like int SIZE, are treated as constants and used for things like defining static array sizes.

Q198. Which of the following syntax correctly defines a default type 'int' for a template parameter 'T'?

A. template <class T = int>

B. template <class T : int>

C. template <T = int>

D. template <int T>

Difficulty: Easy | Importance: 4/5

Explanation:
Default template arguments are specified using the = operator in the template parameter list.

Q199. In partial specialization, what happens to the number of template parameters compared to the general template?

A. It must increase

B. It must decrease

C. It remains the same

D. It becomes zero

Difficulty: Hard | Importance: 4/5

Explanation:
In partial specialization, the number of parameters in the header remains the same, but their nature becomes more specific in the class definition.

Q200. What function is typically used inside the specialized isEqual function for const char* comparisons?

A. ==

B. strcmp

C. strcpy

D. length

Difficulty: Easy | Importance: 5/5

Explanation:
For C-style strings (const char*), direct comparison == compares addresses, so strcmp is used to compare content.

Q201. Why is an explicit insert function required for Vector<char*> instead of just using operator[] for assignment?

A. To handle dynamic memory allocation

B. To increase speed

C. Because operator[] cannot be overloaded

D. To allow deep copying

Difficulty: Hard | Importance: 4/5

Explanation:
Direct assignment via [] can cause memory leaks or errors with char*; insert safely deletes old memory and allocates new memory.

Q202. Can a normal non-template class contain a member function that is a template?

A. No, never

B. Yes, always

C. Only if the class is virtual

D. Yes, these are called member templates

Difficulty: Medium | Importance: 3/5

Explanation:
Regular classes can have member functions that are templates to handle different data types.

Q203. What does the syntax Vector< > v; imply if the class is defined as template <class T = int> class Vector?

A. Syntax Error

B. Creates Vector of void

C. Creates Vector of int

D. Creates Vector of char

Difficulty: Medium | Importance: 5/5

Explanation:
The empty angle brackets < > signal the compiler to use the default template argument, which is int.

Q204. In the ComplexSet example, why is the insert function defined as a template?

A. To accept only int types

B. To accept any instantiation of Complex class

C. To return a Complex object

D. To sort the set

Difficulty: Medium | Importance: 3/5

Explanation:
Making insert a template allows the ComplexSet to store Complex objects of various types (int, float, double).

Q205. Which specialization is considered 'Partial'?

A. template <> class A<int, float>

B. template <class T> class A<T*>

C. template <class T> class A<T>

D. template <class T, class U> class A<T, U>

Difficulty: Hard | Importance: 5/5

Explanation:
A<T*> is a partial specialization because it specializes the behavior for pointers while still keeping T generic.

Q206. In the destructor of Vector<char*>, what must be deleted first?

A. The array pointer ptr

B. The size variable

C. The individual character arrays ptr[i]

D. The object itself

Difficulty: Medium | Importance: 4/5

Explanation:
To prevent memory leaks, you must first delete the individual strings allocated at each index before deleting the main array pointer.

Q207. If a template is defined as template <class T, int I = 5>, what is 'I'?

A. A type parameter

B. A default non-type parameter

C. A class constructor

D. A global variable

Difficulty: Easy | Importance: 4/5

Explanation:
I is a non-type parameter with a default value of 5, treated as a constant within the template.

Q208. What is the main advantage of using non-type parameters for array sizes in templates?

A. Dynamic memory allocation

B. Static memory allocation at compile time

C. Faster sorting

D. Easier debugging

Difficulty: Medium | Importance: 3/5

Explanation:
Passing size as a non-type parameter allows creating arrays on the stack with sizes known at compile time, avoiding new/delete.

Q209. In the context of function templates, template <class T> void func(int, T, 7) is an example of:

A. Complete Specialization

B. Partial Specialization

C. Overloading

D. Default Arguments

Difficulty: Hard | Importance: 4/5

Explanation:
It partially specializes the function by fixing some parameters (int, 7) while leaving others generic (T).

Q210. What is the return type of operator[] in the specialized Vector<char*> class?

A. char

B. char*

C. const char*&

D. void

Difficulty: Hard | Importance: 3/5

Explanation:
It returns a reference to a constant char pointer (const char*&) to allow read access while protecting the pointer address.

Q211. When implementing operator= for Vector<char*>, what is the first step?

A. Copy data

B. Check for self-assignment

C. Allocate new memory

D. Delete old memory

Difficulty: Easy | Importance: 5/5

Explanation:
Standard practice in assignment operators is to check if (this == &right) to avoid corrupting data during self-assignment.

Q212. Which template parameter type allows passing values like 10 or 20 directly in the template argument list?

A. Type parameter

B. Non-type parameter

C. Virtual parameter

D. Abstract parameter

Difficulty: Easy | Importance: 4/5

Explanation:
Non-type parameters allow passing constant values (integers, pointers) rather than data types.

Q213. How does the compiler treat code generation for member templates?

A. Generates all possible types

B. Generates only used instantiations

C. Generates random types

D. Does not generate code

Difficulty: Medium | Importance: 5/5

Explanation:
To avoid code bloat, the compiler only generates machine code for the specific template instantiations that are actually called in the program.

Q214. In function template overloading, which version of a function does the compiler prioritize first?

A. Generic Template

B. Partial Specialization

C. Complete Specialization

D. Ordinary Function

Difficulty: Medium | Importance: 5/5

Explanation:
If an ordinary (non-template) function matches the call exactly, it is preferred over any template version.

Q215. Given Vector<T>, Vector<T*>, and Vector<char*>, which definition is used for Vector<int*>?

A. Vector<T>

B. Vector<T*>

C. Vector<char*>

D. Vector<int>

Difficulty: Medium | Importance: 5/5

Explanation:
Vector<int*> matches the partial specialization Vector<T*> because it is a pointer type but not char*.

Q216. Why does class B : public A<T> { ... }; result in a compilation error if B is not a template?

A. B must be private

B. T is undefined in B

C. A cannot be a base class

D. Inheritance is forbidden

Difficulty: Medium | Importance: 4/5

Explanation:
Since B is an ordinary class, the template parameter T is unknown and undefined within its scope.

Q217. Can an ordinary class inherit from a complete specialization of a template class (e.g., class B : public A<float*>)?

A. Yes, always

B. No, never

C. Only if B is virtual

D. Only if A is abstract

Difficulty: Easy | Importance: 4/5

Explanation:
A complete specialization (like A<float*>) acts like a regular class type, so an ordinary class can inherit from it.

Q218. When an ordinary function is declared as a friend of a class template, it becomes a friend of:

A. Only the int instantiation

B. Only the float instantiation

C. Every instantiation of that class

D. No instantiation

Difficulty: Medium | Importance: 4/5

Explanation:
An ordinary friend declaration grants access to the private members of all instantiations of the template class.

Q219. In template<class T> class B { friend A<T>; }, if B is instantiated as B<int>, which A is the friend?

A. A<float>

B. A<char>

C. A<int>

D. All instantiations of A

Difficulty: Hard | Importance: 5/5

Explanation:
The friend declaration uses the same type parameter T, so friendship is restricted to the matching instantiation A<int>.

Q220. Which specialization allows a template to behave differently for all pointer types?

A. Complete Specialization

B. Partial Specialization

C. General Template

D. Ordinary Class

Difficulty: Medium | Importance: 4/5

Explanation:
A partial specialization like Vector<T*> allows custom behavior for any pointer type while remaining generic.

Q221. What is the correct syntax for a partial specialization of a class A taking two parameters T and U?

A. template <class T> class A<T, T*>

B. template <> class A<int, float>

C. template <class T, class U> class A

D. class A : public T

Difficulty: Hard | Importance: 4/5

Explanation:
Partial specialization changes the parameters in the class name (e.g., A<T, T*>) while keeping some template parameters.

Q222. If sort(char*) exists as an ordinary function, which function is called by sort("Hello")?

A. template <T> sort(T)

B. template <> sort(char*)

C. Ordinary sort(char*)

D. Runtime Error

Difficulty: Easy | Importance: 5/5

Explanation:
The compiler prefers the ordinary function over any template specialization when the match is exact.

Q223. When deriving from a template class, the derived class must generally accept:

A. No parameters

B. At least as many parameters as the base

C. Fewer parameters than the base

D. Only integer parameters

Difficulty: Medium | Importance: 3/5

Explanation:
To instantiate the base template, the derived class must pass the required type parameters, usually received via its own template definition.

Q224. What happens if the compiler finds no complete or partial specialization matching a template usage?

A. It throws an error

B. It uses the General Template

C. It uses the Ordinary Function

D. It creates a new class

Difficulty: Easy | Importance: 4/5

Explanation:
The general template is the fallback option if no specific specialization matches the required type.

Q225. Can a template class inherit from an ordinary (non-template) class?

A. No

B. Yes

C. Only if private

D. Only if protected

Difficulty: Easy | Importance: 3/5

Explanation:
A template class can inherit from an ordinary class; the base class is simply treated as a fixed type.

Q226. Which of the following represents a complete specialization for Vector of char*?

A. template <T> class Vector<T>

B. template <> class Vector<char*>

C. template <class T> class Vector<T*>

D. class Vector

Difficulty: Medium | Importance: 5/5

Explanation:
Complete specialization uses template <> and specifies the exact type char* in the class name.

Q227. In the context of friend functions, what does the rule 'friend void func(T)' imply inside 'class B<T>'?

A. Friendship is universal

B. Friendship is one-to-one based on T

C. Friendship is invalid

D. Func is a static member

Difficulty: Hard | Importance: 4/5

Explanation:
The specific instantiation func(int) is a friend of B<int>, and func(float) is a friend of B<float>.

Q228. In the resolution order example, Vector<int> intVector; will eventually resolve to which implementation?

A. Vector<char*>

B. Vector<T*>

C. Vector<T>

D. Vector<int> (Complete)

Difficulty: Medium | Importance: 4/5

Explanation:
Since no complete specialization for int and no partial specialization for non-pointers exists, it falls back to the general Vector<T>.

Q229. What allows Vector<char*> to have a different implementation (like deep copying) than Vector<int>?

A. Inheritance

B. Polymorphism

C. Template Specialization

D. Encapsulation

Difficulty: Medium | Importance: 5/5

Explanation:
Specialization allows redefining the entire class implementation for specific types to handle cases like C-strings correctly.

Q230. Can a partial specialization inherit from another partial specialization?

A. Yes

B. No

C. Only for int types

D. Only in Java

Difficulty: Medium | Importance: 3/5

Explanation:
The text explicitly confirms that a partial specialization may inherit from another partial specialization.

Q231. What is the term for defining multiple function templates with different parameters or specializations?

A. Function Overriding

B. Function Template Overloading

C. Operator Overloading

D. Dynamic Binding

Difficulty: Easy | Importance: 4/5

Explanation:
Defining specialized versions or different signatures of function templates is known as function template overloading.

Q232. If class B is derived from A<float*>, does B need to be a template class?

A. Yes

B. No

C. Maybe

D. Depends on compiler

Difficulty: Medium | Importance: 4/5

Explanation:
Since A<float*> is a complete type (fully specified), B does not need to be a template to inherit from it.

Q233. In the resolution order for class templates, which specialization does the compiler search for first?

A. General Template

B. Partial Specialization

C. Complete Specialization

D. Abstract Template

Difficulty: Medium | Importance: 5/5

Explanation:
The compiler looks for a Complete Specialization first, then Partial, and finally the General Template.

Q234. When resolving a function call, which type of function takes the highest precedence over templates?

A. Function Template

B. Partial Specialization

C. Ordinary Function

D. Complete Specialization

Difficulty: Easy | Importance: 5/5

Explanation:
Non-template (ordinary) functions are always preferred over template versions if they match the arguments.

Q235. How are static data members handled in template classes?

A. One copy for all types

B. Separate copy for each instantiation

C. No static members allowed

D. Shared by inheritance

Difficulty: Medium | Importance: 5/5

Explanation:
Each instantiation of a template class (e.g., A<int>, A<char>) has its own separate copy of static variables.

Q236. Which implementation will be used for Vector<int*> if Vector<T>, Vector<T*>, and Vector<char*> exist?

A. Vector<T>

B. Vector<T*>

C. Vector<char*>

D. Vector<int>

Difficulty: Hard | Importance: 5/5

Explanation:
Vector<int*> matches the partial specialization Vector<T*> more specifically than the general Vector<T>.

Q237. Which operator is NOT required for the generic find algorithm discussed in the text?

A. Increment (++)

B. Dereference ()

C. Inequality (!=)

D. Modulo (%)

Difficulty: Medium | Importance: 3/5

Explanation:
The find algorithm relies on ++ to traverse, * to access, and != to check bounds, but does not use modulo.

Q238. Why does class B : public A<T> { ... } cause a compiler error if B is an ordinary class?

A. A cannot be base

B. T is undefined in B

C. B must be abstract

D. Inheritance cycle detected

Difficulty: Medium | Importance: 4/5

Explanation:
In an ordinary class, the template parameter T is unknown, so it cannot be passed to the base class A<T>.

Q239. If an ordinary function is declared as a friend of a class template, it is a friend of:

A. Only int instantiation

B. Only char instantiation

C. All instantiations

D. No instantiation

Difficulty: Easy | Importance: 4/5

Explanation:
An ordinary friend function has access to the private members of every instantiation of that template class.

Q240. In template<class T> class B { friend A<T>; }, what is the friendship relationship?

A. One-to-One

B. One-to-Many

C. Many-to-Many

D. No relation

Difficulty: Hard | Importance: 5/5

Explanation:
Since both use the same parameter T, A<int> is only a friend of B<int>, creating a one-to-one mapping.

Q241. What happens if friend class A uses a different template parameter than class B (e.g., template<class U> friend class A)?

A. Compilation Error

B. One-to-One Friendship

C. Universal Friendship

D. Friendship is ignored

Difficulty: Hard | Importance: 4/5

Explanation:
Each instantiation of the friend class becomes a friend of every instantiation of the granting class.

Q242. If A<int> obj1 and A<int> obj2 exist, how many copies of static member data exist?

A. Zero

B. One

C. Two

D. Three

Difficulty: Medium | Importance: 4/5

Explanation:
Objects of the same instantiation share the same static data members.

Q243. What is a major limitation of the basic generic find algorithm implementation?

A. It uses too much RAM

B. It supports only single traversal

C. It cannot handle integers

D. It is recursive

Difficulty: Medium | Importance: 3/5

Explanation:
The iterator/pointer approach used allows moving forward but typically does not support complex or multiple traversal strategies easily.

Q244. Can a partial specialization inherit from another class template?

A. Yes

B. No

C. Only if virtual

D. Only if private

Difficulty: Easy | Importance: 3/5

Explanation:
Partial specializations can inherit from general class templates or other specializations.

Q245. If no complete or partial specialization matches a request, what does the compiler use?

A. Generates an error

B. Uses General Template

C. Uses Ordinary Function

D. Stops compilation

Difficulty: Easy | Importance: 4/5

Explanation:
The general template serves as the fallback definition when no specific specialization matches.

Q246. Can an ordinary class inherit from a complete specialization like A<float*>?

A. Yes

B. No

C. Only in C#

D. Only if abstract

Difficulty: Easy | Importance: 4/5

Explanation:
A complete specialization is a fully defined type, so an ordinary class can inherit from it.

Q247. Defining multiple function templates with different parameters is called:

A. Function Overriding

B. Function Template Overloading

C. Class Specialization

D. Operator Overloading

Difficulty: Medium | Importance: 4/5

Explanation:
This allows different template functions to coexist and be selected based on argument types.

Q248. When is memory reserved for static members of a template class?

A. At program start

B. When the class is instantiated

C. During compilation only

D. Never

Difficulty: Medium | Importance: 4/5

Explanation:
Static members are created and initialized when the specific template instantiation (e.g., A<int>) is generated.

Q249. In the context of templates, what is 'Writability'?

A. Ability to write to disk

B. Ease of writing code once for multiple types

C. Writing generic algorithms only

D. Documentation quality

Difficulty: Easy | Importance: 3/5

Explanation:
Templates improve writability by allowing a single code block to handle multiple data types, reducing code duplication.

Q250. Why might isEqual(str1, str2) fail for char* in a generic template?

A. It compares content

B. It compares pointers (addresses)

C. It throws exception

D. It deletes memory

Difficulty: Medium | Importance: 5/5

Explanation:
The default == operator on pointers compares memory addresses, not the string content.

Q251. If a template is declared as a friend, which of its specializations become friends?

A. Only explicit

B. Only implicit

C. Only partial

D. All kinds (explicit, implicit, partial)

Difficulty: Hard | Importance: 4/5

Explanation:
Declaring a template as a friend grants friendship to all its variations and specializations.

Q252. What is the main drawback of using simple C++ pointers (cursors) for traversing all types of containers?

A. They consume too much memory

B. They fail with non-contiguous containers

C. They cannot be incremented

D. They are always slower than arrays

Difficulty: Medium | Importance: 5/5

Explanation:
Cursors (pointers) rely on contiguous memory for increment operations (++), which fails for non-contiguous structures like linked lists or sets.

Q253. Which three methods must a container provide to support traversal using the 'Cursor' approach?

A. begin(), end(), iterate()

B. first(), beyond(), next()

C. start(), stop(), move()

D. input(), output(), process()

Difficulty: Medium | Importance: 4/5

Explanation:
The text specifies that for the cursor approach, the aggregate object must provide first(), beyond(), and next() methods.

Q254. What is a primary advantage of using Iterators over Cursors?

A. Iterators expose private data

B. Iterators allow multiple pending traversals

C. Iterators are faster than pointers

D. Iterators only work for integers

Difficulty: Medium | Importance: 5/5

Explanation:
Iterators allow multiple independent traversals on a single container simultaneously and support different traversal strategies.

Q255. In the context of STL, what are the three key components?

A. Classes, Objects, Methods

B. Containers, Iterators, Algorithms

C. Inputs, Processes, Outputs

D. Templates, Macros, Functions

Difficulty: Easy | Importance: 5/5

Explanation:
The Standard Template Library (STL) consists of three main components: Containers, Iterators, and Algorithms.

Q256. Which STL container is best described as a 'Doubly linked list'?

A. Vector

B. Deque

C. List

D. Array

Difficulty: Easy | Importance: 5/5

Explanation:
The STL list container is implemented as a doubly linked list, allowing rapid insertions and deletions anywhere.

Q257. Which sequence container provides rapid insertions and deletions at both the front and back?

A. Vector

B. List

C. Deque

D. Stack

Difficulty: Medium | Importance: 4/5

Explanation:
A deque (double-ended queue) allows rapid insertion and deletion at both the front and the back.

Q258. What characteristic distinguishes an STL vector from an STL list?

A. Vector allows random access; List does not

B. List allows random access; Vector does not

C. Vector is a linked list

D. List uses contiguous memory

Difficulty: Medium | Importance: 5/5

Explanation:
Vectors support random access (indexing), while Lists (linked lists) require sequential access.

Q259. In the Iterator class implementation shown, what does the operator* function return?

A. The index number

B. The container size

C. Reference to the element

D. The memory address

Difficulty: Medium | Importance: 3/5

Explanation:
The dereference operator (operator*) returns the actual element (reference) that the iterator is currently pointing to.

Q260. What is the return value of iv.capacity() in the STL Vector example after inserting 3 elements (1, 2, 3, 4) where capacity doubles?

A. 2

B. 3

C. 4

D. 8

Difficulty: Hard | Importance: 3/5

Explanation:
In the example, capacity doubles from 2 to 4 when the 3rd and 4th elements are added.

Q261. Which method is used to add an element to the end of an STL vector?

A. add()

B. insert()

C. push_back()

D. append()

Difficulty: Easy | Importance: 5/5

Explanation:
The standard method to append an element to the end of an STL vector is push_back().

Q262. Why is the cursor approach considered to violate data hiding principles?

A. It uses pointers

B. It exposes internal data implementation

C. It is too slow

D. It uses templates

Difficulty: Medium | Importance: 4/5

Explanation:
Cursors often require accessing the internal memory layout (like pointers to arrays), bypassing encapsulation.

Q263. What does the beyond() method return in the context of Cursors or Iterators?

A. The last element

B. One position past the last element

C. The first element

D. Null pointer

Difficulty: Easy | Importance: 4/5

Explanation:
beyond() returns a pointer/iterator to the position just after the last element, serving as a sentinel for loops.

Q264. Which STL container supports push_front()?

A. Vector

B. Deque

C. Stack

D. Queue

Difficulty: Medium | Importance: 4/5

Explanation:
The deque container supports push_front(), whereas vector only supports push_back().

Q265. Which component of STL helps in traversing a container without exposing its internal representation?

A. Algorithm

B. Cursor

C. Iterator

D. Allocator

Difficulty: Easy | Importance: 5/5

Explanation:
Iterators provide a standard way to traverse containers while maintaining data abstraction and encapsulation.

Q266. If Vector is used as a template parameter for Iterator<CT, ET>, what does CT represent?

A. Container Type

B. Element Type

C. Counter Type

D. Class Type

Difficulty: Medium | Importance: 3/5

Explanation:
In the template definition template<class CT, class ET>, CT represents the Container Type (e.g., Vector).

Q267. In the Generic Iterator example, what is the purpose of the bool pointAtFirst parameter in the constructor?

A. To enable debugging

B. To decide start or beyond position

C. To check if container is empty

D. To validate the pointer

Difficulty: Hard | Importance: 3/5

Explanation:
It determines whether the iterator should initialize pointing to the first element (true) or the beyond position (false).

Q268. What is the result of dq.pop_front() in an STL Deque?

A. Removes the last element

B. Removes the first element

C. Returns the first element without removing

D. Clears the deque

Difficulty: Easy | Importance: 4/5

Explanation:
pop_front() removes the element at the beginning of the deque.

Q269. Which of the following is NOT a Sequence Container in STL?

A. Vector

B. List

C. Deque

D. Map

Difficulty: Medium | Importance: 5/5

Explanation:
Vector, List, and Deque are sequence containers. Map is an Associative Container.

Q270. What allows Generic Algorithms to work on different containers like Vector and Arrays?

A. Inheritance

B. Polymorphism

C. Operator Overloading and Templates

D. Virtual Functions

Difficulty: Hard | Importance: 4/5

Explanation:
Generic algorithms use templates and expect containers to implement standard operators (++, *, !=) to function uniformly.

Q271. Which type of STL container provides fast retrieval of data based on keys?

A. Sequence Containers

B. Associative Containers

C. Container Adapters

D. Derived Containers

Difficulty: Easy | Importance: 5/5

Explanation:
Associative containers (like maps and sets) allow fast retrieval of elements using keys rather than sequential traversal.

Q272. Which of the following is considered a 'First-class Container' in STL?

A. Stack

B. Queue

C. Vector

D. Priority Queue

Difficulty: Medium | Importance: 5/5

Explanation:
Sequence containers (like Vector) and Associative containers are collectively referred to as first-class containers, whereas Stack is an adapter.

Q273. What type of iterator is supported by the STL list container?

A. Random Access Iterator

B. Bidirectional Iterator

C. Forward Iterator

D. Input Iterator

Difficulty: Medium | Importance: 5/5

Explanation:
Lists allow traversal in both directions (forward and backward) but do not support random access (indexing).

Q274. Which STL container adapter follows the LIFO (Last In First Out) principle?

A. Queue

B. Priority Queue

C. Stack

D. Vector

Difficulty: Easy | Importance: 5/5

Explanation:
A stack is a container adapter that operates on the Last In First Out (LIFO) principle.

Q275. Which operation is supported by Random Access Iterators but NOT by Bidirectional Iterators?

A. Increment (++p)

B. Dereference (*p)

C. Subscript operator (p[i])

D. Equality comparison (==)

Difficulty: Hard | Importance: 5/5

Explanation:
Random Access Iterators allow direct access to elements using the subscript operator [], which bidirectional iterators do not support.

Q276. What does the end() function of a container return?

A. Iterator to the last element

B. Iterator to the first element

C. Iterator to the position beyond the last element

D. A null pointer

Difficulty: Medium | Importance: 4/5

Explanation:
The end() function returns an iterator referring to the position just past the last element, used as a sentinel.

Q277. Which STL container allows duplicate keys?

A. set

B. map

C. multiset

D. multimap

Difficulty: Medium | Importance: 4/5

Explanation:
While map requires unique keys, multimap allows multiple elements to have the same key.

Q278. Do Container Adapters (stack, queue, priority_queue) support iterators?

A. Yes, all of them

B. Yes, but only stack

C. No, they do not

D. Only priority_queue does

Difficulty: Medium | Importance: 5/5

Explanation:
Container adapters are constrained versions of containers and do not allow free traversal of elements via iterators.

Q279. What distinguishes a deque from a vector?

A. Deque allows insertions at both ends

B. Deque uses less memory

C. Deque is an associative container

D. Deque has no random access

Difficulty: Medium | Importance: 4/5

Explanation:
A deque (double-ended queue) supports rapid insertions and deletions at both the front and the back.

Q280. Which function returns an iterator to the last element of a container (reverse beginning)?

A. end()

B. begin()

C. rbegin()

D. rend()

Difficulty: Medium | Importance: 3/5

Explanation:
rbegin() returns a reverse iterator pointing to the last element of the container.

Q281. In an associative container, what happens if you try to retrieve an element using a formula like Value % 10?

A. You must traverse the whole list

B. You move directly to the required element

C. It causes a compilation error

D. It creates a new element

Difficulty: Easy | Importance: 3/5

Explanation:
Associative containers use keys/formulas to jump directly to the element's location without sequential traversal.

Q282. What is the return value of the max_size() function?

A. The current number of elements

B. The maximum number of elements the container can hold

C. The size of one element in bytes

D. The memory address of the container

Difficulty: Easy | Importance: 3/5

Explanation:
max_size() returns the theoretical limit on the number of elements a container can hold.

Q283. Which iterator category has the least capabilities?

A. Random Access Iterator

B. Bidirectional Iterator

C. Forward Iterator

D. Input/Output Iterator

Difficulty: Medium | Importance: 4/5

Explanation:
Input and Output iterators are the most basic, allowing only single-pass sequential access, while Random Access are the most powerful.

Q284. Which STL container is implemented as a Doubly Linked List?

A. Vector

B. Deque

C. List

D. Map

Difficulty: Easy | Importance: 5/5

Explanation:
The std::list container is a doubly linked list, which allows bidirectional traversal but no random access.

Q285. In the context of std::priority_queue, which element is always returned?

A. The first element inserted

B. The last element inserted

C. The element with the highest priority

D. A random element

Difficulty: Medium | Importance: 4/5

Explanation:
Priority queues order elements such that the highest priority element is always accessible at the top.

Q286. What does the clear() function do to a container?

A. Resets the iterator

B. Deletes the container object

C. Erases all elements from the container

D. Sorts the elements

Difficulty: Easy | Importance: 4/5

Explanation:
clear() removes all elements from the container, leaving it with a size of 0.

Q287. Can a vector use a Random Access Iterator?

A. Yes

B. No

C. Only if sorted

D. Only for char types

Difficulty: Easy | Importance: 5/5

Explanation:
Vectors support random access (indexing), so they use Random Access Iterators.

Q288. Which underlying containers can std::queue adapt?

A. Vector or List

B. Deque or List

C. Vector or Deque

D. Stack or List

Difficulty: Hard | Importance: 3/5

Explanation:
The queue adapter can be built on top of a deque or a list to provide FIFO functionality.

Q289. Which type of iterator is supported by the std::vector container?

A. Forward Iterator

B. Bidirectional Iterator

C. Random Access Iterator

D. Input Iterator

Difficulty: Easy | Importance: 5/5

Explanation:
Vectors store elements in contiguous memory, allowing direct access to any element, thus supporting Random Access Iterators.

Q290. Which of the following is an example of a Mutating-Sequence Algorithm?

A. find

B. count

C. fill

D. accumulate

Difficulty: Medium | Importance: 4/5

Explanation:
Mutating-Sequence Algorithms, like fill, change the values or positions of elements in a container.

Q291. In the context of STL, std::set supports which type of iterator?

A. Random Access Iterator

B. Bidirectional Iterator

C. Output Iterator

D. Input Iterator

Difficulty: Medium | Importance: 5/5

Explanation:
Sets allow traversal in both forward and backward directions but do not support random access via index.

Q292. Which block in exception handling contains the code that is suspected to cause an error?

A. catch

B. throw

C. try

D. finally

Difficulty: Easy | Importance: 5/5

Explanation:
The try block encloses the code that might generate an exception, allowing it to be caught by a subsequent block.

Q293. Approximately how many standard algorithms does the STL include?

A. 50

B. 70

C. 100

D. 25

Difficulty: Easy | Importance: 3/5

Explanation:
The text states that STL includes 70 standard algorithms for manipulating containers.

Q294. Which of the following is classified as a Numeric Algorithm in STL?

A. copy

B. find

C. accumulate

D. generate

Difficulty: Medium | Importance: 4/5

Explanation:
accumulate is listed under Numeric Algorithms, which involve mathematical calculations.

Q295. What is a major disadvantage of the 'Return Error Code' technique for error handling?

A. It crashes the system immediately

B. It increases program complexity and reduces readability

C. It automatically saves data

D. It separates error logic from main logic

Difficulty: Medium | Importance: 5/5

Explanation:
Mixing error handling code (checking return values) with main logic makes the code less readable and more complex.

Q296. Which keyword is used to raise an exception in C++?

A. raise

B. try

C. catch

D. throw

Difficulty: Easy | Importance: 5/5

Explanation:
The throw keyword is used to signal that an exception has occurred, followed by an exception object or value.

Q297. Why should primitive data types be avoided when throwing exceptions?

A. They are too slow

B. They consume too much memory

C. They can cause ambiguity

D. They cannot be caught

Difficulty: Medium | Importance: 4/5

Explanation:
Using primitive types can lead to ambiguity regarding the nature of the error; defining new classes is preferred.

Q298. What happens during 'Abnormal Termination' of a program?

A. The program saves all data

B. The Operating System terminates the program

C. The program retries the operation

D. Local cleanup is performed

Difficulty: Medium | Importance: 4/5

Explanation:
In abnormal termination, the OS kills the process due to an error (like divide by zero) without saving data.

Q299. Which iterator operation is INVALID for an ostream_iterator (Output Iterator)?

A. outputIt = value

B. outputIt++

C. x = outputIt

D. All are valid

Difficulty: Hard | Importance: 3/5

Explanation:
Output iterators are for writing (assigning to L-value); reading from them (x = *outputIt) causes an error.

Q300. Which of the following is a Non-Mutating Sequence Algorithm?

A. copy

B. fill

C. count

D. partition

Difficulty: Medium | Importance: 4/5

Explanation:
Non-Mutating algorithms, like count and find, do not change the elements or their positions in the container.

Q301. In the 'Graceful Termination' technique, what function is typically called after local cleanup?

A. abort()

B. terminate()

C. return 0

D. exit()

Difficulty: Medium | Importance: 3/5

Explanation:
The text demonstrates using exit(1) to terminate the program gracefully after detecting an error and performing cleanup.

Q302. Which syntax is correct for throwing a mathematical expression as an exception?

A. throw 1+29;

B. throw {1+29};

C. throw [1+29];

D. throw func(1+29);

Difficulty: Medium | Importance: 3/5

Explanation:
The text confirms that any expression, including mathematical ones like throw 1+2*9;, can be used to represent an exception.

Q303. What is the main benefit of Exception Handling over other error handling techniques?

A. It consumes less memory

B. It separates main logic from error handling code

C. It prevents all runtime errors

D. It is faster to execute

Difficulty: Medium | Importance: 5/5

Explanation:
Exception handling allows the main logic to remain clean by delegating error processing to catch blocks.

Q304. Which algorithm is used to fill a range with a specific value?

A. copy

B. generate

C. fill

D. partition

Difficulty: Easy | Importance: 4/5

Explanation:
The fill algorithm assigns a specific value to every element in a specified range of a container.

Q305. Which of the following falls under the category of Numeric Algorithms?

A. find_first_of

B. inner_product

C. equal

D. iter_swap

Difficulty: Medium | Importance: 3/5

Explanation:
inner_product, accumulate, partial_sum, and adjacent_difference are classified as Numeric Algorithms.

Q306. In the 'Return Illegal Value' error handling technique, what is a potential risk?

A. System crash due to illegal value usage

B. Compilation error

C. Infinite loop

D. Memory leak

Difficulty: Medium | Importance: 4/5

Explanation:
Returning an illegal value might be ignored or misused by the calling function, potentially leading to a system crash later.

Q307. Which statement about std::istream_iterator (Input Iterator) is true?

A. It can be used for writing data

B. It supports multi-pass algorithms

C. It is used for reading elements in a single pass

D. It supports random access

Difficulty: Medium | Importance: 4/5

Explanation:
Input iterators are designed for reading data sequentially in a forward direction, typically in a single pass.

Q308. Which keyword is used to signal that an exceptional condition has occurred in a C++ program?

A. catch

B. try

C. throw

D. return

Difficulty: Easy | Importance: 5/5

Explanation:
The 'throw' keyword is used to raise an exception when an error occurs.

Q309. In C++ exception handling, catch handlers are differentiated based on what?

A. Return type

B. Argument type

C. Function name

D. Line number

Difficulty: Medium | Importance: 5/5

Explanation:
Catch blocks are selected based on the data type of the argument passed to them (e.g., int, float, or a class object).

Q310. What happens to local objects in an executing block during stack unwinding?

A. They are preserved

B. They are destroyed

C. They are copied to heap

D. They are converted to static

Difficulty: Medium | Importance: 5/5

Explanation:
During stack unwinding, all local objects in the path of the exception are destroyed (their destructors are called).

Q311. Does stack unwinding automatically destroy dynamically allocated memory (pointers)?

A. Yes, always

B. No, never

C. Only if null

D. Only in debug mode

Difficulty: Hard | Importance: 4/5

Explanation:
Stack unwinding only destroys stack-allocated local objects; memory allocated with 'new' is not automatically freed.

Q312. Which function is called by default if no catch handler matches a thrown exception?

A. main()

B. exit()

C. terminate()

D. return()

Difficulty: Medium | Importance: 4/5

Explanation:
If an exception is not caught, the C++ runtime calls the 'terminate()' function, which typically calls 'abort()'.

Q313. What is the correct syntax to catch ALL exceptions regardless of their type?

A. catch(all)

B. catch(...)

C. catch(ANY)

D. catch(void)

Difficulty: Easy | Importance: 5/5

Explanation:
The syntax 'catch(...)' with three dots (ellipsis) is used to catch any exception thrown.

Q314. Why is it recommended to pass exceptions by reference in a catch handler?

A. To save memory

B. To avoid shallow copy problems

C. To allow modification

D. To increase speed

Difficulty: Medium | Importance: 4/5

Explanation:
Passing by reference avoids object slicing and shallow copy issues, preserving the exact type and state of the exception object.

Q315. How does the compiler evaluate multiple catch blocks?

A. Randomly

B. In reverse order

C. In the order they are written

D. Based on best fit

Difficulty: Medium | Importance: 4/5

Explanation:
Catch blocks are checked sequentially; the first one that matches the exception type is executed.

Q316. What happens if you place catch(...) as the first catch block?

A. It catches everything, making subsequent blocks unreachable

B. It causes a syntax error

C. It is ignored

D. It only catches integers

Difficulty: Hard | Importance: 4/5

Explanation:
Since catch(...) matches any exception, placing it first prevents any specific handlers below it from ever executing.

Q317. Which statement allows a function to partially handle an exception and then pass it up the stack?

A. return;

B. throw;

C. continue;

D. goto catch;

Difficulty: Medium | Importance: 5/5

Explanation:
A bare 'throw;' statement (without an operand) inside a catch block re-throws the current exception to the next level.

Q318. In the context of exception handling, what is 'Stack Unwinding'?

A. Expanding the stack memory

B. Returning control and destroying local objects

C. Creating a new stack

D. Allocating dynamic memory

Difficulty: Medium | Importance: 5/5

Explanation:
Stack unwinding is the process of exiting function scopes and destroying local variables as control searches for a catch handler.

Q319. Can a try block exist without a catch block?

A. Yes

B. No

C. Only in main

D. Only for void functions

Difficulty: Easy | Importance: 3/5

Explanation:
A try block must be followed by at least one catch handler (or catch(...)).

Q320. If an exception is thrown from an inner try block and its immediate catch does not match, where does control go?

A. Program crashes

B. It goes to the outer catch blocks

C. It returns to main

D. It ignores the exception

Difficulty: Medium | Importance: 4/5

Explanation:
The runtime searches outward through nested try-catch blocks to find a matching handler.

Q321. Which feature allows catching a single exception type for a whole category of errors?

A. Polymorphism

B. Inheritance

C. Templates

D. Overloading

Difficulty: Medium | Importance: 3/5

Explanation:
By creating a hierarchy of exception classes, you can catch a base class type to handle all derived exception types.

Q322. What is thrown by the statement throw DivideByZero();?

A. A function pointer

B. An anonymous object of class DivideByZero

C. A string

D. An integer

Difficulty: Easy | Importance: 3/5

Explanation:
The syntax constructs a temporary (anonymous) object of the class DivideByZero and throws it.

Q323. In the example catch(DivideByZero & obj), what is the scope of obj?

A. Global

B. Entire main function

C. Only within that catch block

D. Throughout the class

Difficulty: Easy | Importance: 3/5

Explanation:
The exception object declared in the catch handler is local to that specific catch block.

Q324. What is a major benefit of using Exception Handling over return codes?

A. It makes code smaller

B. It separates error handling code from program logic

C. It prevents logic errors

D. It runs faster

Difficulty: Easy | Importance: 4/5

Explanation:
Exception handling allows the main flow of the program to remain clean, while error logic is moved to catch blocks.

Q325. Can primitive types like int or float be thrown as exceptions?

A. No, only objects

B. Yes, but it is discouraged

C. Only if they are constant

D. Only in void functions

Difficulty: Medium | Importance: 3/5

Explanation:
C++ allows throwing primitives (e.g., throw 1;), but it is discouraged because it can cause ambiguity.

Q326. If you have catch(MathError) and catch(DivideByZero) where DivideByZero inherits from MathError, which should come first?

A. MathError

B. DivideByZero

C. It does not matter

D. catch(...)

Difficulty: Hard | Importance: 5/5

Explanation:
The most specific handler (DivideByZero) must appear before the general handler (MathError), or it will never be reached.

Q327. When is the object thrown as an exception destroyed?

A. Immediately after throw

B. When the catch handler completes execution

C. It is never destroyed

D. When the program ends

Difficulty: Medium | Importance: 4/5

Explanation:
The exception object lives until the matching catch handler finishes its execution.

Q328. If an exception is thrown before fclose(fileptr) is executed in a standard function, what is the result?

A. File closes automatically

B. Resource leak occurs

C. Program restarts

D. Compilation error

Difficulty: Easy | Importance: 5/5

Explanation:
The call to fclose is skipped due to the exception, leaving the file open and causing a resource leak.

Q329. How can resource wastage be prevented when an exception is thrown?

A. Using global variables

B. Using a class with a destructor to release resources

C. Using goto statements

D. Ignoring the exception

Difficulty: Medium | Importance: 5/5

Explanation:
Objects allocated on the stack are destroyed during stack unwinding, so a destructor can safely release resources like file pointers.

Q330. If an exception is thrown inside a class constructor, which of the following is TRUE regarding the object's destructor?

A. It is called immediately

B. It is never called

C. It is called after main()

D. It is called by the compiler

Difficulty: Medium | Importance: 5/5

Explanation:
The destructor for the object itself is not called because the object was not fully constructed.

Q331. What happens to the data members that were already fully constructed if the constructor throws an exception?

A. They remain in memory

B. Their destructors are called

C. They become global

D. They cause a memory leak

Difficulty: Hard | Importance: 4/5

Explanation:
For any fully constructed members within the object, their destructors are invoked automatically before the exception propagates.

Q332. Where can an exception thrown by a member initialization list be caught?

A. Inside the constructor body

B. In the main function only

C. In a function-try block wrapping the constructor

D. It cannot be caught

Difficulty: Hard | Importance: 3/5

Explanation:
To catch exceptions from the initialization list, the constructor must use a function-try block syntax try : member(val) { ... }.

Q333. Why should an exception NOT be allowed to leave a destructor?

A. It is a syntax error

B. It causes infinite loops

C. It may collide with an active stack unwinding

D. It uses too much memory

Difficulty: Medium | Importance: 5/5

Explanation:
If a destructor is called during stack unwinding and it throws another exception, C++ calls terminate() because it cannot handle two active exceptions.

Q334. What function is called by default if an exception escapes a destructor during stack unwinding?

A. std::abort()

B. std::exit()

C. std::terminate()

D. std::unexpected()

Difficulty: Medium | Importance: 5/5

Explanation:
The runtime calls std::terminate(), which in turn usually calls std::abort(), crashing the program.

Q335. What does the function signature void func() throw() imply?

A. Function can throw any exception

B. Function cannot throw any exception

C. Function throws only int

D. Function throws only char

Difficulty: Easy | Importance: 5/5

Explanation:
An empty throw specification list throw() indicates that the function is not allowed to throw any exceptions.

Q336. If a function void func() is declared without a throw specification, what exceptions can it throw?

A. No exceptions

B. Only standard exceptions

C. Any exception

D. Only integers

Difficulty: Easy | Importance: 4/5

Explanation:
In C++ (pre-C++11), omitting the exception specification means the function can throw any type of exception.

Q337. Which function is called if a function throws an exception not listed in its exception specification?

A. std::terminate()

B. std::unexpected()

C. std::abort()

D. std::exit()

Difficulty: Medium | Importance: 4/5

Explanation:
Violating the exception specification triggers the unexpected() function.

Q338. To handle cases where a function violates its throw list, a programmer can set a custom handler using:

A. set_terminate

B. set_unexpected

C. set_handler

D. set_exception

Difficulty: Hard | Importance: 3/5

Explanation:
The set_unexpected function allows the programmer to define a custom handler for unexpected exceptions.

Q339. In the context of exception handling, what is 'Stack Unwinding'?

A. Allocating memory on stack

B. Destroying local objects as control exits scopes

C. Copying stack to heap

D. Increasing stack size

Difficulty: Medium | Importance: 5/5

Explanation:
Stack unwinding is the process of exiting function scopes and destroying local variables while searching for a catch block.

Q340. Which feature allows defining a list of exceptions a function is allowed to throw?

A. Exception Specification

B. Template Specialization

C. Operator Overloading

D. Function Overriding

Difficulty: Easy | Importance: 4/5

Explanation:
Exception specification (using the throw(...) syntax) restricts the types of exceptions a function can propagate.

Q341. Which C++ concept ensures that an object is automatically cleaned up when it goes out of scope?

A. Polymorphism

B. Encapsulation

C. Destructor

D. Inheritance

Difficulty: Easy | Importance: 5/5

Explanation:
The destructor is automatically called when an object goes out of scope, making it ideal for resource management (RAII).